

```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyEstate.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S16_Main";
    }
    /// <summary>
    /// S00_Boot scene host: register mock services -> run seven steps -> switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// </summary>
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null && !op.isDone) yield return null;
        }
    }
}

// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.CrazyEstate.Boot
{
    /// <summary>
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &= Run(1, "Logger", LoggerStep);
        ok &= Run(2, "Time", TimeStep);
        ok &= Run(3, "Config", ConfigStep);
        ok &= Run(4, "Save", SaveStep);
        ok &= Run(5, "ILocale", LocaleStep);
        ok &= Run(6, "Ads.Preload", AdsStep);
        ok &= Run(7, "SwitchMain", () =&gt; _switchScene != null && _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 && !time.TimeCheated();
    }
    static bool ConfigStep()
    {

```

```
        var config = ServiceLocator.Get<IConfigService>();
        var update = config.CheckUpdate("mock://manifest");
        return update.NewVersion != null; // mock = no hot update
    }
    static bool SaveStep()
    {
        var save = ServiceLocator.Get<ISaveService>();
        save.Save("boot_probe", "{}");
        return !string.IsNullOrEmpty(save.Load("boot_probe"));
    }
    static bool LocaleStep()
    {
        var locale = ServiceLocator.Get<ILocaleService>();
        return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
    }
    static bool AdsStep()
    {
        ServiceLocator.Get<IAdsService>().Preload("boot_warmup");
        return true;
    }
}

// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyEstate.Boot
{
    /// <summary>
    /// Dev-time mock service set (SY MOCK scope). All seven boot steps depend on this set.
    /// </summary>
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet<ITimeService>(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register<ITimeService>(new MockTimeService());
            ServiceLocator.Register<IConfigService>(new MockConfigService());
            ServiceLocator.Register<ISaveService>(new MockSaveService());
            ServiceLocator.Register<ILocaleService>(new MockLocaleService());
            ServiceLocator.Register<IAdsService>(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY MOCK)");
        }
    }

    public sealed class MockTimeService : ITimeService
    {

```

```
public TimeSource Source => TimeSource.Mock;
public long ClientNowMs() => DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
/// <summary>SY MOCK: device time doubles as server time until the real chain lands.</summary>
public long ServerNowMs() => ClientNowMs();
public bool TimeCheated() => Math.Abs(ServerNowMs() - ClientNowMs()) > 120_000;
}

public sealed class MockConfigService : IConfigService
{
    public T GetTable<T>(string tableId) where T : class
        => throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable<T>(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        => new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) => "mock-0.0.1";
}

public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) => PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) => PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// <summary>No cloud: local storage is the authority (zero-server doctrine).</summary>
    public SyncResult CloudSync() => new SyncResult(false, null);
}

public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// <summary>Missing key falls back to the key itself with a warning.</summary>
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        => new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}

public sealed class MockAdsService : IAdsService
{
    public event Action<AdLifecycle> OnAdEvent;
    public Task<RewardResult> ShowRewarded(string pointId)
        => Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
    {
        Logger.Info("ADS", "preload (mock): " + pointId);
    }
}
```

```
        OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
    }
}
}
// AboutPanel16.cs
// G16 About disclosure panel (X1271 T017 slice, commitment-letter items 5+6:
// platform AIGC labeling rule + random-content static disclosure - the G11
// WorkerAboutPanel X420 paradigm on the G16 desk-family chassis). Every
// disclosure line composes FROM the live law tables: the lottery odds format
// straight out of LotterySim's frozen bp anchors and the ad caps out of the
// shared AdCapPolicy tables, so the published numbers can never drift from
// the enforced ones. The composer layer is pure static; the headless
// AboutCheck asserts it without rendering. Programmatic UGUI, zero scene-YAML
// drift, presentation-only. Boot-hidden like every desk host so the
// redline-11 hook timeline holds.
using System.Globalization;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.CrazyEstate
{
    public sealed class AboutPanel16 : MonoBehaviour
    {
        public const string KeyTitle = "about_title";
        public const string KeyAigcLine = "about_aigc_line";
        public const string KeyOddsLine = "about_odds_line";
        public const string KeyAdCapLine = "about_adcap_line";
        public const string KeyAgeLine = "about_age_line";
        public const string KeyCloseLabel = "about_close";
        static readonly Color TextLight = new Color(0.95f, 0.96f, 0.98f, 1f);
        static readonly Color TextDim = new Color(0.62f, 0.66f, 0.72f, 1f);
        public int Taps;
        RectTransform _overlay;
        Button _openButton;
        // ----- composer layer (pure static, gate-facing) -----
        /// <summary>AIGC explicit disclosure: art AND audio assets in this
        /// game are AI-assisted. Downgrade law (G11 parity): should the
        /// audio face ever return to procedural synthesis, this copy drops
        /// the audio half the same batch - two versions never ship.</summary>
        public static string AigcLine()
        {
            return CopyTable.Get(KeyAigcLine);
        }
        /// <summary>Hot-lot odds disclosure. The published numbers format
        /// straight out of LotterySim's frozen anchors (base 3000bp,
        /// +750bp per on-beat tap, 6000bp cap) - the shown face and the
        /// enforced draw law are the same object, the same construction
        /// guarantee the in-window live "lottery_chance" face uses.</summary>
        public static string OddsLine()
        {
            return string.Format(CultureInfo.InvariantCulture, CopyTable.Get(KeyOddsLine),
```

```
BpPct(LotterySim.BaseWinBp), BpPct(LotterySim.OnBeatBonusBp),
BpPct(LotterySim.WinCapBp));
}
/// <summary>Ad-cap disclosure: the three G16 category ceilings
/// formatted out of the live AdCapPolicy tables (g16_rescue 5 /
/// g16_speed 8 / g16_double 1) plus the 12/day global cap -
/// voluntary views only, no forced ad.</summary>
public static string AdCapLine()
{
    return string.Format(CultureInfo.InvariantCulture, CopyTable.Get(KeyAdCapLine),
        Sy.Platform.AdCapPolicy.CategoryCap(Sy.Platform.AdCategories.G16Rescue),
        Sy.Platform.AdCapPolicy.CategoryCap(Sy.Platform.AdCategories.G16Speed),
        Sy.Platform.AdCapPolicy.CategoryCap(Sy.Platform.AdCategories.G16Double),
        Sy.Platform.AdCapPolicy.DefaultDailyCap);
}
/// <summary>Age-rating disclosure (IAA casual default: 8+; no IAP;
/// the platform backend filing stays an admin item).</summary>
public static string AgeLine()
{
    return CopyTable.Get(KeyAgeLine);
}
public static string PanelTitle()
{
    return CopyTable.Get(KeyTitle);
}
public static string CloseLabel()
{
    return CopyTable.Get(KeyCloseLabel);
}
// bp (1/10000) -> percent string ("30", "7.5") - never hand-packed.
// public: the Editor gate reads it across assemblies (X618 Snapshot law).
public static string BpPct(int bp)
{
    return (bp / 100.0).ToString("0.##", CultureInfo.InvariantCulture);
}
// ----- view layer (programmatic UGUI, zero scene YAML) -----
void Start()
{
    BuildUi();
    SetDeskOpen(false); // boot hidden: redline-11 timeline law
}
void BuildUi()
{
    var canvasGo = new GameObject("AboutCanvas", typeof(Canvas), typeof(CanvasScaler),
        typeof(GraphicRaycaster));
    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = 14; // disclosure modal rides above popup (12) and act show (13)
    var scaler = canvasGo.GetComponent<CanvasScaler>();
    // U184: ScaleWithScreenSize or the 720x1280 reference frame never
```

```
// drives device scale (raw ConstantPixelSize = shrunken UI).
scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
scaler.referenceResolution = new Vector2(720, 1280);
// U262 safe-area law (17 doc section 4, B641 pattern): every
// screen rides one full-stretch SafeArea child squeezed to
// Screen.safeArea (notch/rounded-corner squeeze, zero clip).
canvasGo = SafeAreaRoot16.Mount(canvasGo);
// The open button joins the left desk column (board 560 /
// mortgage 440 / achieve 320 / license 200 / finale 80 / about -40).
_openButton = MakeButton(canvasGo.transform, "AboutOpenButton",
    new Vector2(-260f, -40f), 160, 80);
var openLabel = _openButton.GetComponentInChildren<Text>();
openLabel.fontSize = 26;
openLabel.text = PanelTitle();
_openButton.onClick.AddListener(OnOpenTapped);
_overlay = (RectTransform)new GameObject("AboutOverlay", typeof(RectTransform)).transform;
_overlay.SetParent(canvasGo.transform, false);
_overlay.sizeDelta = new Vector2(720f, 1280f);
var dim = NewImage(_overlay, "Dim");
dim.color = new Color(0.03f, 0.04f, 0.06f, 0.72f);
Stretch(dim.rectTransform);
var back = NewImage(_overlay, "PanelBack");
back.color = new Color(0.08f, 0.11f, 0.16f, 0.96f);
back.rectTransform.anchoredPosition = new Vector2(0f, -40f);
back.rectTransform.sizeDelta = new Vector2(680f, 1000f);
var title = MakeText(_overlay, "AboutTitle", 34f, new Vector2(0f, 400f), 560, 70);
title.text = PanelTitle();
var aigc = MakeText(_overlay, "AigcLine", 24f, new Vector2(0f, 270f), 560, 120);
aigc.text = AigcLine();
aigc.horizontalOverflow = HorizontalWrapMode.Wrap;
aigc.color = TextLight;
var odds = MakeText(_overlay, "OddsLine", 22f, new Vector2(0f, 80f), 560, 220);
odds.text = OddsLine();
odds.horizontalOverflow = HorizontalWrapMode.Wrap;
odds.color = TextLight;
var adcap = MakeText(_overlay, "AdCapLine", 20f, new Vector2(0f, -120f), 560, 150);
adcap.text = AdCapLine();
adcap.horizontalOverflow = HorizontalWrapMode.Wrap;
adcap.color = TextDim;
var age = MakeText(_overlay, "AgeLine", 22f, new Vector2(0f, -290f), 560, 80);
age.text = AgeLine();
age.horizontalOverflow = HorizontalWrapMode.Wrap;
age.color = TextDim;
var close = MakeButton(_overlay, "AboutCloseButton", new Vector2(0f, -405f), 420, 90);
var closeLabel = close.GetComponentInChildren<Text>();
closeLabel.fontSize = 28;
closeLabel.text = CloseLabel();
close.onClick.AddListener(OnCloseTapped);
}
void SetDeskOpen(bool open)
```

```
{
    if (_overlay != null) _overlay.gameObject.SetActive(open);
    if (_openButton != null) _openButton.gameObject.SetActive(!open);
}
void OnOpenTapped()
{
    Taps++;
    SetDeskOpen(true);
}
void OnCloseTapped()
{
    Taps++;
    SetDeskOpen(false);
}
// ----- UGUI helpers (host-local style, no shared dependencies) -----
static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var t = go.GetComponent<Text>();
    t.font = UIFont.Resolve();
    t.fontSize = Mathf.RoundToInt(size);
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    return t;
}
static Button MakeButton(Transform parent, string name, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var img = go.GetComponent<Image>();
    img.color = new Color(0.22f, 0.19f, 0.12f, 1f);
    var btn = go.GetComponent<Button>();
    // One-Graphic-per-GO engine law: Tuanjie 2022.3 refuses a second
    // Graphic on the same GameObject (AddComponent<Text> returns null,
    // X1192 wave-1 proof), so the label rides a stretch-to-parent
    // child GO instead.
    var labelGo = new GameObject("Label", typeof(RectTransform), typeof(Text));
    labelGo.transform.SetParent(go.transform, false);
    var lrt = (RectTransform)labelGo.transform;
    lrt.anchorMin = Vector2.zero;
    lrt.anchorMax = Vector2.one;
    lrt.offsetMin = Vector2.zero;
    lrt.offsetMax = Vector2.zero;
```

```
        var label = labelGo.GetComponent<Text>();
        label.font = UIFont.Resolve();
        label.fontSize = 30;
        label.alignment = TextAnchor.MiddleCenter;
        label.color = Color.white;
        // X1700 AU10: generic tap voice on every programmatic button,
        // registered before the host's semantic listener so the beat
        // lands after the click (order law, census-neutral new key).
        btn.onClick.AddListener(delegate { G16AudioHost.Ensure().PlaySfx("sfx_btn"); });
        return btn;
    }

    static Image NewImage(Transform parent, string name)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(parent, false);
        return go.GetComponent<Image>();
    }

    static void Stretch(RectTransform rt)
    {
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = Vector2.zero;
        rt.offsetMax = Vector2.zero;
    }
}

// AchieveSim.cs
// G16 honor wall achievement set (T012 slice C / requirements doc M07
// section 6 achievement table, SYS09 derive-only law). Eight honors, every
// one DERIVED live from single truths that already exist - the rent ledger
// (first home / city crown / tenant + review codex via the album faces),
// the bargain sim (best single-duel cut), the mortgage ledger (paid-off
// law) and the IAA row (zero-ad law, U091). Nothing is stored twice: the
// wall is a projection, so no save face for it exists by construction.
// Pure C#, zero scene, zero RNG. The wall View host lands with slice D.
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.CrazyEstate
{
    public enum AchieveId
    {
        FirstHome = 1,
        BargainGod = 2,
        RentFreedom = 3,
        ReviewCollector = 4,
        TenantCodex = 5,
        CityCrown = 6,
        ZeroAd = 7,
        Tycoon = 8,
    }
}
```

```

}
public struct AchieveEntry
{
    public AchieveId Id;
    public string Stem;
    public string CopyKey;
    public bool Unlocked;
}
public static class AchieveSim
{
    public const int Total = 8;           // honors on the wall
    public const int BargainGodCutWan = 5; // one duel, five wan off (T013 slice B:
                                           // % band starter duel saves 5 wan = 12.5%
                                           // of list - god at starter scale, the
                                           // first guided deal still crowns)
    public const int CrownUnitId = 10;     // H10 "The World Crown"
    public const string ZeroAdCopyKey = "zero_ad_achievement"; // T010 key reuse
    static readonly AchieveId[] NonTycoonIds =
    {
        AchieveId.FirstHome, AchieveId.BargainGod, AchieveId.RentFreedom,
        AchieveId.ReviewCollector, AchieveId.TenantCodex,
        AchieveId.CityCrown, AchieveId.ZeroAd,
    };
    // Per-honor laws (requirements doc section 6):
    // FirstHome      - keys in hand (any ledger row).
    // BargainGod      - best single-deal cut reaches five wan (slice B
    //                  % band: starter duel 5 wan = 12.5% of list).
    // RentFreedom     - the per-beat rent flow strictly outruns the
    //                  mortgage bill (no bill = instant freedom, comedy).
    // ReviewCollector - the review-literature album face goes full.
    // TenantCodex     - the tenant album face goes full.
    // CityCrown       - the World Crown (H10) sits in the ledger
    //                  (production attainability arrives with the
    //                  H04-H10 listing expansion batch).
    // ZeroAd          - a home paid off installment by installment
    //                  with zero ad grants ever (U091 declaration).
    // Tycoon          - the other seven all shine.
    public static bool IsUnlocked(AchieveId id, RentLedger ledger, BargainSim bargain,
        MortgageSim mortgage, IaaSim iaa, float nowSec)
    {
        switch (id)
        {
            case AchieveId.FirstHome:
                return ledger != null && ledger.Rows.Count > 0;
            case AchieveId.BargainGod:
                return bargain != null && bargain.BestSavingsWan >= BargainGodCutWan;
            case AchieveId.RentFreedom:
                return ledger != null && RentFlowCoins(ledger) > 0
                    && RentFlowCoins(ledger) > DueCoins(mortgage, nowSec);
            case AchieveId.ReviewCollector:

```

```
        return AlbumFull(AlbumSim.ReviewAlbum(ledger));
    case AchieveId.TenantCodex:
        return AlbumFull(AlbumSim.TenantAlbum(ledger));
    case AchieveId.CityCrown:
        if (ledger == null) return false;
        for (var i = 0; i < ledger.Rows.Count; i++)
            if (ledger.Rows[i].UnitId == CrownUnitId) return true;
        return false;
    case AchieveId.ZeroAd:
        return iaa != null && iaa.Row.Grants == 0 && HasPaidOffRow(mortgage);
    case AchieveId.Tycoon:
        for (var i = 0; i < NonTycoonIds.Length; i++)
            if (!IsUnlocked(NonTycoonIds[i], ledger, bargain, mortgage, iaa, nowSec))
                return false;
        return true;
    }
    return false;
}

public static List<AchieveEntry> Derive(RentLedger ledger, BargainSim bargain,
    MortgageSim mortgage, IaaSim iaa, float nowSec)
{
    var entries = new List<AchieveEntry>();
    for (var id = AchieveId.FirstHome; id <= AchieveId.Tycoon; id++)
        entries.Add(new AchieveEntry
        {
            Id = id,
            Stem = StemOf(id),
            CopyKey = CopyKeyOf(id),
            Unlocked = IsUnlocked(id, ledger, bargain, mortgage, iaa, nowSec),
        });
    return entries;
}

public static string StemOf(AchieveId id)
{
    return "ach_" + ((int)id).ToString("D2", CultureInfo.InvariantCulture);
}

public static string CopyKeyOf(AchieveId id)
{
    switch (id)
    {
        case AchieveId.FirstHome: return "ach_first_home";
        case AchieveId.BargainGod: return "ach_bargain_god";
        case AchieveId.RentFreedom: return "ach_rent_freedom";
        case AchieveId.ReviewCollector: return "ach_review_collector";
        case AchieveId.TenantCodex: return "ach_tenant_codex";
        case AchieveId.CityCrown: return "ach_city_crown";
        case AchieveId.ZeroAd: return ZeroAdCopyKey;
        default: return "ach_tycoon";
    }
}
```

```
public static int UnlockedCount(RentLedger ledger, BargainSim bargain,
    MortgageSim mortgage, IaaSim iaa, float nowSec)
{
    var n = 0;
    for (var id = AchieveId.FirstHome; id &lt;= AchieveId.Tycoon; id++)
        if (IsUnlocked(id, ledger, bargain, mortgage, iaa, nowSec)) n++;
    return n;
}
// Wall faces (slice D view consumes; statics stay the gate's
// single truth source - no second truth exists).
// "Honors earned: {0}/{1}"
public static string ComposeWallCount(int unlocked, int total)
{
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("ach_wall_count"), unlocked, total);
}
public static string ComposeWallLine(AchieveEntry entry)
{
    return entry.Unlocked
        ? CopyTable.Get(entry.CopyKey)
        : CopyTable.Get("ach_locked");
}
public static string WallTitle()
{
    return CopyTable.Get("ach_wall_title");
}
// Eight-bit projection of the wall - the determinism face.
public static string WallStamp(RentLedger ledger, BargainSim bargain,
    MortgageSim mortgage, IaaSim iaa, float nowSec)
{
    var sb = new StringBuilder();
    for (var id = AchieveId.FirstHome; id &lt;= AchieveId.Tycoon; id++)
        sb.Append(IsUnlocked(id, ledger, bargain, mortgage, iaa, nowSec) ? '1' : '0');
    return sb.ToString();
}
static int RentFlowCoins(RentLedger ledger)
{
    var flow = 0;
    if (ledger == null) return 0;
    for (var i = 0; i &lt; ledger.Rows.Count; i++)
        flow += RentLedger.RentPerBeat(ledger.Rows[i].UnitId);
    return flow;
}
static int DueCoins(MortgageSim mortgage, float nowSec)
{
    return mortgage == null ? 0 : mortgage.DueCoins(nowSec);
}
static bool AlbumFull(List<AlbumEntry> face)
{
    if (face == null || face.Count == 0) return false;
```

```

        for (var i = 0; i < face.Count; i++)
            if (!face[i].Unlocked) return false;
        return true;
    }
    static bool HasPaidOffRow(MortgageSim mortgage)
    {
        if (mortgage == null) return false;
        for (var i = 0; i < mortgage.Rows.Count; i++)
        {
            var r = mortgage.Rows[i];
            if (!r.Sold && r.LoanCoins == 0 && r.MonthsPaid > 0) return true;
        }
        return false;
    }
    // ----- T020-d five-line finale archive (requirements doc M14: the
    // "Oasis Legends" wall). Same derive-only family as the eight-wall:
    // every face reuses a single truth that already exists - LIC01 rides
    // the existing RentFreedom honor, LIC04 rides CityCrown (H10 brick in
    // the skyline), LIC03 reads the star-storefront ladder's full-house
    // cap. The two new axes mirror the license unlock pattern; initial
    // anchors are frozen here, the bot parameter sweep re-anchors, never
    // a round. Pure C#, zero RNG, the wall is a projection.
    public const int StorageEmpireHoldings = 5; // LIC02 full rank: the top holding tier
    public const int SnipeKingPoolWins = 3;    // LIC05: three pool gavels
    public static int FinaleLineTotal { get { return LicenseTable.LineCount; } }
    // Per-line finale law: small landlord = rent freedom / rent tycoon
    // = storage empire at the top tier / gold agent = the five-star
    // full-house storefront / developer = the H10 crown brick on the
    // skyline / bottom fisher = the snipe king.
    public static bool LineFinaleMet(LicenseId line, RentLedger ledger,
        MortgageSim mortgage, float nowSec, int starSavingsWan, int snipePoolWins)
    {
        switch (line)
        {
            case LicenseId.SmallLandlord:
                return IsUnlocked(AchieveId.RentFreedom, ledger, null, mortgage, null, nowSec);
            case LicenseId.RentTycoon:
                return ledger != null && ledger.Rows.Count >= StorageEmpireHoldings;
            case LicenseId.GoldAgent:
                return StarStoreTable.StarsFor(starSavingsWan) >= StarStoreTable.MaxStars;
            case LicenseId.Developer:
                return IsUnlocked(AchieveId.CityCrown, ledger, null, null, null, nowSec);
            case LicenseId.BottomFisher:
                return snipePoolWins >= SnipeKingPoolWins;
        }
        return false;
    }
    public static int FinaleLitCount(RentLedger ledger, MortgageSim mortgage,
        float nowSec, int starSavingsWan, int snipePoolWins)
    {

```

```

        var n = 0;
        for (var i = 0; i < LicenseTable.Lines.Length; i++)
            if (LineFinaleMet(LicenseTable.Lines[i].Id, ledger, mortgage,
                nowSec, starSavingsWan, snipePoolWins)) n++;
        return n;
    }

    // Five-bit projection in line order 1..5 - the determinism face the
    // finale funnel observes (FinaleTracker consumes this stamp).
    public static string FinaleStamp(RentLedger ledger, MortgageSim mortgage,
        float nowSec, int starSavingsWan, int snipePoolWins)
    {
        var sb = new StringBuilder();
        for (var i = 0; i < LicenseTable.Lines.Length; i++)
            sb.Append(LineFinaleMet(LicenseTable.Lines[i].Id, ledger, mortgage,
                nowSec, starSavingsWan, snipePoolWins) ? '1' : '0');
        return sb.ToString();
    }

    public static string FinaleCopyKeyOf(LicenseId line)
    {
        return "lic_finale_0" + (int)line;
    }
}

// AchieveViewHost.cs
// G16 honor wall view host (T012 slice D). Runtime-built UGUI face for
// S16_Main: the achievement wall - eight derive-only honors projected
// live from the single truths the sims already own (SYS09: the wall is
// a projection, no second truth). The wall binds lazily to the four
// live sources (rent ledger + IAA row via the rent desk, bargain sim
// via the duel host, mortgage sim via the mortgage desk) and tolerates
// any of them missing: an unbound wall renders eight locked lines and
// 0/8, never a blank crash. Boot-hidden so the bargain hook keeps the
// redline-11 timeline (same law as every desk, X623 pattern).
using System.Text;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.CrazyEstate
{
    public sealed class AchieveViewHost : MonoBehaviour
    {
        GameObject _panel;
        Button _openButton;
        Text _countFace;
        readonly Text[] _lineFaces = new Text[AchieveSim.Total];
        RentLedger _ledger;
        BargainSim _bargain;
        MortgageSim _mortgage;
        IaaSim _iaa;
        string _lastBody;
        public int Taps;
    }
}

```

```
public bool DeskOpen { get { return _panel != null && _panel.activeSelf; } }
// ----- Pure decision layer (single truth, shared with AchieveCheck) -----
// Count face: "Honors earned: {n}/{total}" composed from the live
// derive - delegates to the sim, zero second truth.
public static string ComposeCountFace(RentLedger ledger, BargainSim bargain,
    MortgageSim mortgage, IaaSim iaa, float nowSec)
{
    return AchieveSim.ComposeWallCount(AchieveSim.UnlockedCount(
        ledger, bargain, mortgage, iaa, nowSec), AchieveSim.Total);
}
// Wall body: one line per honor, id order top to bottom, each line
// the sim's own compose (unlocked = honor copy, locked = locked copy).
public static string ComposeWallBody(RentLedger ledger, BargainSim bargain,
    MortgageSim mortgage, IaaSim iaa, float nowSec)
{
    var sb = new StringBuilder();
    var entries = AchieveSim.Derive(ledger, bargain, mortgage, iaa, nowSec);
    for (var i = 0; i < entries.Count; i++)
    {
        if (i > 0) sb.Append('\n');
        sb.Append(AchieveSim.ComposeWallLine(entries[i]));
    }
    return sb.ToString();
}
// Shine-vs-dim face law: earned honors glow gold (warm family),
// locked honors sit neutral gray (r == g == b) - the wall reads
// at one glance (AC15).
public static Color LineTint(bool unlocked)
{
    return unlocked ? new Color(1.00f, 0.82f, 0.35f)
        : new Color(0.48f, 0.48f, 0.48f);
}
void Start()
{
    BuildUi();
    SetDeskOpen(false); // boot hidden: redline-11 timeline law
}
void Update()
{
    TryBind();
    if (!DeskOpen) return;
    RefreshFaces();
}
// Lazy bind: the four live sources come from other hosts' Starts, so
// retry per frame is order-immune (derive tolerates nulls meanwhile).
void TryBind()
{
    if (_ledger == null)
    {
        var desk = FindObjectOfType<RentViewHost>();
    }
}
```

```
        if (desk != null && desk.Ledger != null)
        {
            _ledger = desk.Ledger;
            _iaa = desk.Iaa;
        }
    }
    if (_bargain == null)
    {
        var duel = FindObjectOfType<BargainViewHost>();
        if (duel != null) _bargain = duel.Sim;
    }
    if (_mortgage == null)
    {
        var mortgageDesk = FindObjectOfType<MortgageViewHost>();
        if (mortgageDesk != null) _mortgage = mortgageDesk.Sim;
    }
}
// ----- Live faces -----
void RefreshFaces()
{
    var body = ComposeWallBody(_ledger, _bargain, _mortgage, _iaa, Time.time);
    var count = ComposeCountFace(_ledger, _bargain, _mortgage, _iaa, Time.time);
    if (body == _lastBody && _countFace != null && _countFace.text == count) return;
    _lastBody = body;
    if (_countFace != null) _countFace.text = count;
    var lines = body.Split('\n');
    for (var i = 0; i < _lineFaces.Length && i < lines.Length; i++)
    {
        if (_lineFaces[i] == null) continue;
        _lineFaces[i].text = lines[i];
        // The stamp face drives the tint: a line lights when its id
        // bit is set in the same derive the gate reads (AC13 face).
        var id = (AchieveId)(i + 1);
        var unlocked = AchieveSim.IsUnlocked(id, _ledger, _bargain,
            _mortgage, _iaa, Time.time);
        _lineFaces[i].color = LineTint(unlocked);
    }
}
// ----- UI construction (programmatic, zero scene authoring) -----
void BuildUi()
{
    var canvasGo = new GameObject("AchieveCanvas", typeof(Canvas), typeof(CanvasScaler),
        typeof(GraphicRaycaster));
    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = 4; // the wall sits under every desk (6-10)
    var scaler = canvasGo.GetComponent<CanvasScaler>();
    // U184: ScaleWithScreenSize or the 720x1280 reference frame never
    // drives device scale (raw ConstantPixelSize = shrunken UI on phone).
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
```

```

    scaler.referenceResolution = new Vector2(720, 1280);
    // U262 safe-area law (17 doc section 4, B641 pattern): every
    // screen rides one full-stretch SafeArea child squeezed to
    // Screen.safeArea (notch/rounded-corner squeeze, zero clip).
    canvasGo = SafeAreaRoot16.Mount(canvasGo);
    _openButton = MakeButton(canvasGo.transform, "AchieveOpenButton",
        new Vector2(-260f, 320f), 160, 80);
    _openButton.GetComponentInChildren<Text>().fontSize = 24;
    _openButton.GetComponentInChildren<Text>().text = AchieveSim.WallTitle();
    _openButton.onClick.AddListener(OnOpenTapped);
    _panel = new GameObject("AchievePanel", typeof(RectTransform), typeof(Image));
    _panel.transform.SetParent(canvasGo.transform, false);
    var prt = (RectTransform)_panel.transform;
    prt.sizeDelta = new Vector2(700, 940);
    prt.anchoredPosition = new Vector2(0f, -60f);
    var img = _panel.GetComponent<Image>();
    img.color = new Color(0.08f, 0.11f, 0.16f, 0.96f);
    var title = MakeText(_panel.transform, "WallTitle", 44f, new Vector2(0f, 410f), 660, 70);
    title.text = AchieveSim.WallTitle();
    _countFace = MakeText(_panel.transform, "WallCount", 30f, new Vector2(0f, 330f), 660, 60);
    _countFace.color = LineTint(true);
    for (var i = 0; i < AchieveSim.Total; i++)
        _lineFaces[i] = MakeText(_panel.transform, "WallLine" + (i + 1),
            26f, new Vector2(0f, 240f - i * 70f), 660, 60);
}

static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var t = go.GetComponent<Text>();
    t.font = UIFont.Resolve();
    t.fontSize = Mathf.RoundToInt(size);
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    return t;
}

static Button MakeButton(Transform parent, string name, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var img = go.GetComponent<Image>();
    img.color = new Color(0.22f, 0.19f, 0.12f, 1f);
    var btn = go.GetComponent<Button>();
    // One-Graphic-per-GO engine law: Tuanjie 2022.3 refuses a second

```

```

        // Graphic on the same GameObject (GetComponent<Text> returns null,
        // X1192 wave-1 proof), so the label rides a stretch-to-parent
        // child GO instead.
        var labelGo = new GameObject("Label", typeof(RectTransform), typeof(Text));
        labelGo.transform.SetParent(go.transform, false);
        var lrt = (RectTransform)labelGo.transform;
        lrt.anchorMin = Vector2.zero;
        lrt.anchorMax = Vector2.one;
        lrt.offsetMin = Vector2.zero;
        lrt.offsetMax = Vector2.zero;
        var label = labelGo.GetComponent<Text>();
        label.font = UIFont.Resolve();
        label.fontSize = 30;
        label.alignment = TextAnchor.MiddleCenter;
        label.color = Color.white;
        // X1700 AU10: generic tap voice on every programmatic button,
        // registered before the host's semantic listener so the beat
        // lands after the click (order law, census-neutral new key).
        btn.onClick.AddListener(delegate { G16AudioHost.Ensure().PlaySfx("sfx_btn"); });
        return btn;
    }
    // ----- Input -----
    void SetDeskOpen(bool open)
    {
        _panel.SetActive(open);
        _openButton.gameObject.SetActive(!open);
        if (open) RefreshFaces();
    }
    void OnOpenTapped()
    {
        Taps++;
        SetDeskOpen(true);
    }
}

// AlbumSim.cs
// G16 codex album + share (T009 slice A / requirements doc M10 + SYS09,
// X032 offline shape 2: everything composes locally, zero server, zero
// upload - the payload IS the product). Four albums feed the codex face:
// property / tenant / event / review-literature. Unlock state is always
// DERIVED, never stored twice - the property and tenant faces read the
// rent ledger rows, the event face reads the event stream codex
// collection, and the review-literature face mirrors the tenant face (a
// review is collectible exactly when its tenant is on the book). The
// share dual channel composes from live data: a short share card and the
// "Tycoon Annual Report" long-image payload (title + six stat lines, all
// copy through CopyTable so both locale faces stay the single authority).
// Each successful share walks exactly one evt_share_card event carrying
// the card_id; an empty portfolio refuses both channels and emits nothing
// (nothing worth flexing yet - the comedy is the point).

```

```
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using Sy.Telemetry;
namespace Sy.CrazyEstate
{
    public struct AlbumEntry
    {
        public string Stem;
        public bool Unlocked;
    }
    public static class AlbumSim
    {
        public const string CardIdShare = "share_card"; // channel 1: short card
        public const string CardIdReport = "annual_report"; // channel 2: report long image
        // --- four albums (SYS09): derive-only, never a second truth ---
        // Property album: one entry per listing, unlocked when a ledger row
        // owns that listing id.
        public static List<AlbumEntry> PropertyAlbum(RentLedger ledger)
        {
            var owned = new HashSet<int>();
            if (ledger != null)
                for (var i = 0; i < ledger.Rows.Count; i++) owned.Add(ledger.Rows[i].UnitId);
            var entries = new List<AlbumEntry>();
            for (var i = 0; i < ListingTable.Listings.Length; i++)
            {
                var l = ListingTable.Listings[i];
                entries.Add(new AlbumEntry
                {
                    Stem = "prop_" + l.Id.ToString("D2", CultureInfo.InvariantCulture),
                    Unlocked = owned.Contains(l.Id),
                });
            }
            return entries;
        }
        // Tenant album: one entry per roster seat, unlocked when any ledger
        // row carries that deterministic tenant id.
        public static List<AlbumEntry> TenantAlbum(RentLedger ledger)
        {
            var met = new HashSet<int>();
            if (ledger != null)
                for (var i = 0; i < ledger.Rows.Count; i++) met.Add(ledger.Rows[i].TenantId);
            var entries = new List<AlbumEntry>();
            for (var i = 0; i < TenantTable.Tenants.Length; i++)
            {
                var t = TenantTable.Tenants[i];
                entries.Add(new AlbumEntry
                {
                    Stem = "tenant_" + t.Id.ToString("D2", CultureInfo.InvariantCulture),
                    Unlocked = met.Contains(t.Id),
                });
            }
        }
    }
}
```

```
        });
    }
    return entries;
}

// Event album: mirrors the event stream codex collection (E01-E03 S1).
public static List<AlbumEntry> EventAlbum(EventStream stream)
{
    var entries = new List<AlbumEntry>();
    for (var i = 0; i < EventCardTable.Cards.Length; i++)
    {
        var c = EventCardTable.Cards[i];
        entries.Add(new AlbumEntry
        {
            Stem = "evt_e" + ((int)c.Id).ToString("D2", CultureInfo.InvariantCulture),
            Unlocked = stream != null && stream.IsCollected(c.Id),
        });
    }
    return entries;
}

// Review-literature album: the tenant face mirrored - a tenant's
// review is collectible exactly when the tenant is on the book.
public static List<AlbumEntry> ReviewAlbum(RentLedger ledger)
{
    var tenantFace = TenantAlbum(ledger);
    var entries = new List<AlbumEntry>();
    for (var i = 0; i < tenantFace.Count; i++)
        entries.Add(new AlbumEntry
        {
            Stem = "review_" + (i + 1).ToString("D2", CultureInfo.InvariantCulture),
            Unlocked = tenantFace[i].Unlocked,
        });
    return entries;
}

// --- share dual channel (X032 offline shape 2) -----
// Channel 2 payload: the "Tycoon Annual Report" long image - title
// plus six live stat lines (homes, portfolio value, lifetime rent,
// tenants, event codex, market mood).
public static string ComposeAnnualReport(RentLedger ledger, EventStream stream, MarketCycle cycle)
{
    var owned = ledger == null ? 0 : ledger.Rows.Count;
    var valueWan = 0;
    var lifetimeCoins = 0;
    var tenants = new HashSet<int>();
    if (ledger != null)
    {
        for (var i = 0; i < ledger.Rows.Count; i++)
        {
            var row = ledger.Rows[i];
            valueWan += ListingTable.GetListing(row.UnitId).ListPriceWan;
            tenants.Add(row.TenantId);
        }
    }
}
```

```
    }
    lifetimeCoins = ledger.TotalCollectedCoins + ledger.PendingCoins;
}
var events = stream == null ? 0 : stream.CollectedCount;
var sb = new StringBuilder();
sb.Append(CopyTable.Get("share_report_title"));
sb.Append('\n').Append(string.Format(CultureInfo.InvariantCulture,
    CopyTable.Get("report_line_owned"), owned));
sb.Append('\n').Append(string.Format(CultureInfo.InvariantCulture,
    CopyTable.Get("report_line_value"), valueWan));
sb.Append('\n').Append(string.Format(CultureInfo.InvariantCulture,
    CopyTable.Get("report_line_rent"), lifetimeCoins));
sb.Append('\n').Append(string.Format(CultureInfo.InvariantCulture,
    CopyTable.Get("report_line_tenants"), tenants.Count));
sb.Append('\n').Append(string.Format(CultureInfo.InvariantCulture,
    CopyTable.Get("report_line_events"), events, EventCardTable.Count));
sb.Append('\n').Append(string.Format(CultureInfo.InvariantCulture,
    CopyTable.Get("report_line_market"),
    CopyTable.Get(cycle == null ? "market_flat" : cycle.PhaseNameKey)));
return sb.ToString();
}
// Channel 1 payload: the short share card. Null on an empty
// portfolio - nothing worth flexing yet.
public static string ComposeShareCard(RentLedger ledger)
{
    if (ledger == null || ledger.Rows.Count == 0) return null;
    var owned = ledger.Rows.Count;
    var lifetimeCoins = ledger.TotalCollectedCoins + ledger.PendingCoins;
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("share_card"), owned, lifetimeCoins);
}
// Share a composed payload through the telemetry funnel: exactly
// one evt_share_card per successful share, carrying the card_id. An
// empty portfolio refuses and emits nothing.
public static bool Share(RentLedger ledger, TelemetryService telemetry, string cardId)
{
    if (ledger == null || ledger.Rows.Count == 0) return false;
    if (telemetry != null)
        telemetry.Track(EventIds.ShareCard, new Dictionary<string, string>
        {
            ["card_id"] = cardId,
        });
    return true;
}
}
}
// AlbumViewHost.cs
// G16 codex album desk view host (T009 slice-B). Runtime-built UGUI face
// for S16_Main: the codex browsing desk - four category tabs (property /
// tenant / event / review-literature) over the derive-only AlbumSim
```

```
// faces, plus the offline share dual channel (short card / "Tycoon Annual
// Report"). Share payloads compose locally (X032 offline shape 2: the
// payload IS the product, zero server), land on the system clipboard and
// walk exactly one evt_share_card event each (AlbumSim single truth).
// T023-G3: the short-card channel also flexes a visual face - the X1198
// card_share_base catalog art under the live payload text on a
// tap-to-close modal (overlay text = the channel-1 payload itself, zero
// second truth).
// Live sources are pulled at open time - ledger from the rent desk,
// stream from the event popup host, cycle from the market HUD - so no
// Start-order race can ever show a stale face. The desk panel stays
// hidden at boot so the bargain hook keeps the redline-11 timeline.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using Sy.Core;
using Sy.Telemetry;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.CrazyEstate
{
    public sealed class AlbumViewHost : MonoBehaviour
    {
        public const int CategoryCount = 4;
        static readonly string[] CatKeys =
        {
            "album_cat_property", "album_cat_tenant", "album_cat_event", "album_cat_review",
        };
        GameObject _panel;
        Button _openButton;
        readonly Button[] _tabButtons = new Button[CategoryCount];
        Text _statusFace;
        Text _bodyFace;
        Button _cardButton;
        Button _reportButton;
        Text _toastFace;
        GameObject _cardFaceRoot;
        Image _cardFaceArt;
        Text _cardFaceText;
        RentLedger _ledger;
        EventStream _stream;
        MarketCycle _cycle;
        TelemetryService _telemetry;
        public int Taps;
        public int CategoryIndex;
        public string LastCardIdShared;
        public bool DeskOpen { get { return _panel != null && _panel.activeSelf; } }
        // ----- Pure decision layer (single truth, shared with AlbumCheck) -----
        // T023-G3: catalog stem of the share-card base art (X1198 landing).
```

```
public const string CardFaceStem = "card_share_base";
// The card overlay text reads the channel-1 payload verbatim - the
// sim stays the single composer, null refuse on an empty portfolio.
public static string ComposeCardFaceText(RentLedger ledger)
{
    return AlbumSim.ComposeShareCard(ledger);
}
// Category tab face: cat copy plus the live unlock count.
public static string ComposeTabFace(string catKey, int unlocked, int total)
{
    var inv = CultureInfo.InvariantCulture;
    return CopyTable.Get(catKey) + " " + unlocked.ToString(inv) + "/" + total.ToString(inv);
}
// One body line per entry: the resolved name when unlocked, the
// locked comedy copy otherwise.
public static string ComposeEntryLine(AlbumEntry entry, string name)
{
    return entry.Unlocked ? name : CopyTable.Get("album_locked");
}
// Full category body: one line per entry, top to bottom.
public static string ComposeAlbumBody(List<AlbumEntry> entries, Func<int, string> nameResolver)
{
    var sb = new StringBuilder();
    for (var i = 0; i < entries.Count; i++)
    {
        if (i > 0) sb.Append('\n');
        sb.Append(ComposeEntryLine(entries[i], nameResolver(i)));
    }
    return sb.ToString();
}
// Status line: the blank-codex copy until anything anywhere unlocks.
public static string ComposeStatusFace(bool anyUnlocked)
{
    return anyUnlocked ? string.Empty : CopyTable.Get("album_empty");
}
// Share pair law: nothing worth flexing until the portfolio exists.
public static bool ShareButtonsVisible(RentLedger ledger)
{
    return ledger != null && ledger.Rows.Count > 0;
}
// Offline share (compose + funnel walk, zero server): the payload
// composes from live data, then the sim decides the single event.
public static bool TryShare(string cardId, RentLedger ledger, EventStream stream,
    MarketCycle cycle, TelemetryService telemetry, out string payload)
{
    payload = null;
    if (!ShareButtonsVisible(ledger)) return false;
    payload = cardId == AlbumSim.CardIdReport
        ? AlbumSim.ComposeAnnualReport(ledger, stream, cycle)
        : AlbumSim.ComposeShareCard(ledger);
}
```

```
        return AlbumSim.Share(ledger, telemetry, cardId);
    }
    // Name resolvers: one per category, CopyTable the single authority.
    static string ResolvePropertyName(int index)
    {
        return CopyTable.Get(ListingTable.Listings[index].HouseKey);
    }
    static string ResolveTenantName(int index)
    {
        return CopyTable.Get(TenantTable.Tenants[index].NameKey);
    }
    static string ResolveEventName(int index)
    {
        return CopyTable.Get(EventCardTable.Cards[index].TitleKey);
    }
    static string ResolveReviewName(int index)
    {
        return CopyTable.Get(TenantTable.Tenants[index].ReviewKey);
    }
    void Start()
    {
        _telemetry = new TelemetryService();
        _telemetry.AddSink(new DebugLogSink()); // local funnel face until the platform sink slice
        BuildUi();
        SetDeskOpen(false);
    }
    // Live single-truth pull at open time - Start order never matters.
    void ResolveSources()
    {
        var rent = FindObjectOfType<RentViewHost>();
        if (rent != null) _ledger = rent.Ledger;
        var events = FindObjectOfType<EventPopupHost>();
        if (events != null) _stream = events.Stream;
        var market = FindObjectOfType<MarketViewHost>();
        if (market != null) _cycle = market.Cycle;
    }
    // ----- Live faces -----
    List<AlbumEntry> FaceFor(int category)
    {
        switch (category)
        {
            case 0: return AlbumSim.PropertyAlbum(_ledger);
            case 1: return AlbumSim.TenantAlbum(_ledger);
            case 2: return AlbumSim.EventAlbum(_stream);
            default: return AlbumSim.ReviewAlbum(_ledger);
        }
    }
    static Func<int, string> ResolverFor(int category)
    {
        switch (category)
```

```
{
    case 0: return ResolvePropertyName;
    case 1: return ResolveTenantName;
    case 2: return ResolveEventName;
    default: return ResolveReviewName;
}
}
bool AnyUnlockedAcrossFaces()
{
    for (var c = 0; c < CategoryCount; c++)
    {
        var face = FaceFor(c);
        for (var i = 0; i < face.Count; i++)
            if (face[i].Unlocked) return true;
    }
    return false;
}
void RefreshAlbumFace()
{
    if (_tabButtons[0] == null) return;
    for (var c = 0; c < CategoryCount; c++)
    {
        var face = FaceFor(c);
        var unlocked = 0;
        for (var i = 0; i < face.Count; i++) if (face[i].Unlocked) unlocked++;
        if (_tabButtons[c] != null)
            _tabButtons[c].GetComponent<Text>().text =
                ComposeTabFace(CatKeys[c], unlocked, face.Count);
    }
    if (_bodyFace != null)
        _bodyFace.text = ComposeAlbumBody(FaceFor(CategoryIndex), ResolverFor(CategoryIndex));
    if (_statusFace != null)
        _statusFace.text = ComposeStatusFace(AnyUnlockedAcrossFaces());
    var shareVisible = ShareButtonsVisible(_ledger);
    if (_cardButton != null) _cardButton.gameObject.SetActive(shareVisible);
    if (_reportButton != null) _reportButton.gameObject.SetActive(shareVisible);
}
// ----- UI construction (programmatic, zero scene authoring) -----
void BuildUi()
{
    var canvasGo = new GameObject("AlbumCanvas", typeof(Canvas), typeof(CanvasScaler),
        typeof(GraphicRaycaster));
    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = 6; // codex desk under every desk (7-9) and the board (10)
    var scaler = canvasGo.GetComponent<CanvasScaler>();
    // U184: ScaleWithScreenSize or the 720x1280 reference frame never
    // drives device scale (raw ConstantPixelSize = shrunken UI on phone).
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(720, 1280);
}
```

```

// U262 safe-area law (17 doc section 4, B641 pattern): every
// screen rides one full-stretch SafeArea child squeezed to
// Screen.safeArea (notch/rounded-corner squeeze, zero clip).
canvasGo = SafeAreaRoot16.Mount(canvasGo);
_openButton = MakeButton(canvasGo.transform, "AlbumOpenButton", new Vector2(260f, 200f), 160, 80);
_openButton.GetComponentInChildren<Text>().text = CopyTable.Get("album_open");
_openButton.onClick.AddListener(OnOpenTapped);
_panel = new GameObject("AlbumPanel", typeof(RectTransform), typeof(Image));
_panel.transform.SetParent(canvasGo.transform, false);
var prt = (RectTransform)_panel.transform;
prt.sizeDelta = new Vector2(700, 940);
prt.anchoredPosition = new Vector2(0f, -60f);
var img = _panel.GetComponent<Image>();
img.color = new Color(0.09f, 0.13f, 0.18f, 0.96f);
var title = MakeText(_panel.transform, "AlbumTitle", 44f, new Vector2(0f, 410f), 660, 70);
title.text = CopyTable.Get("album_title");
_statusFace = MakeText(_panel.transform, "StatusFace", 26f, new Vector2(0f, 330f), 660, 50);
_statusFace.text = string.Empty;
var tabXs = new[] { -255f, -85f, 85f, 255f };
for (var i = 0; i < CategoryCount; i++)
{
    var idx = i;
    _tabButtons[i] = MakeButton(_panel.transform, "Tab" + idx, new Vector2(tabXs[idx], 250f), 160, 70);
    _tabButtons[i].GetComponentInChildren<Text>().fontSize = 22;
    _tabButtons[i].onClick.AddListener(delegate { OnTabTapped(idx); });
}
_bodyFace = MakeText(_panel.transform, "AlbumBody", 24f, new Vector2(0f, -40f), 660, 520);
_bodyFace.alignment = TextAnchor.UpperLeft;
_cardButton = MakeButton(_panel.transform, "ShareCardButton", new Vector2(-170f, -350f), 320, 84);
_cardButton.GetComponentInChildren<Text>().fontSize = 26;
_cardButton.GetComponentInChildren<Text>().text = CopyTable.Get("share_open");
_cardButton.onClick.AddListener(OnShareCardTapped);
_reportButton = MakeButton(_panel.transform, "ShareReportButton", new Vector2(170f, -350f), 320, 84);
_reportButton.GetComponentInChildren<Text>().fontSize = 20;
_reportButton.GetComponentInChildren<Text>().text = CopyTable.Get("share_report_title");
_reportButton.onClick.AddListener(OnShareReportTapped);
_toastFace = MakeText(_panel.transform, "ShareToast", 26f, new Vector2(0f, -430f), 660, 50);
_toastFace.text = string.Empty;
BuildCardFaceUi(canvasGo.transform);
}
// T023-G3 share-card visual face: full-stretch dim backdrop that
// closes on any tap, the card_share_base art centered at its native
// 890x1200 aspect, and the live payload over the art's calm bottom
// band (QC safe zone 80-97%). Hidden at boot so the redline-11
// timeline never sees it; art resolves lazily at open time so a
// missing sprite degrades to a text-on-dim face, never a crash.
void BuildCardFaceUi(Transform parent)
{
    _cardFaceRoot = new GameObject("ShareCardFace", typeof(RectTransform), typeof(Image), typeof(Button));
    _cardFaceRoot.transform.SetParent(parent, false);

```

```
var brt = (RectTransform)_cardFaceRoot.transform;
brt.anchorMin = Vector2.zero;
brt.anchorMax = Vector2.one;
brt.offsetMin = Vector2.zero;
brt.offsetMax = Vector2.zero;
var backdrop = _cardFaceRoot.GetComponent<Image>();
backdrop.color = new Color(0f, 0f, 0f, 0.72f);
_cardFaceRoot.GetComponent<Button>().onClick.AddListener(CloseCardFace);
var cardGo = new GameObject("CardArt", typeof(RectTransform), typeof(Image));
cardGo.transform.SetParent(_cardFaceRoot.transform, false);
var crt = (RectTransform)cardGo.transform;
crt.sizeDelta = new Vector2(540f, 728f); // 890x1200 base art aspect
crt.anchoredPosition = new Vector2(0f, 20f);
_cardFaceArt = cardGo.GetComponent<Image>();
_cardFaceArt.preserveAspect = true;
_cardFaceArt.raycastTarget = false;
_cardFaceText = MakeText(cardGo.transform, "CardFaceText", 26f, new Vector2(0f, -280f), 500, 110);
var shadow = _cardFaceText.gameObject.AddComponent<Shadow>();
shadow.effectColor = new Color(0f, 0f, 0f, 0.65f);
shadow.effectDistance = new Vector2(2f, -2f);
_cardFaceRoot.SetActive(false);
}
void OpenCardFace(string payload)
{
    if (_cardFaceRoot == null || string.IsNullOrEmpty(payload)) return;
    if (_cardFaceArt != null) _cardFaceArt.sprite = ArtRuntime.Get(CardFaceStem);
    if (_cardFaceText != null) _cardFaceText.text = payload;
    _cardFaceRoot.SetActive(true);
}
void CloseCardFace()
{
    if (_cardFaceRoot != null) _cardFaceRoot.SetActive(false);
}
static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var t = go.GetComponent<Text>();
    t.font = UIFont.Resolve();
    t.fontSize = Mathf.RoundToInt(size);
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    return t;
}
static Button MakeButton(Transform parent, string name, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
```

```
go.transform.SetParent(parent, false);
var rt = (RectTransform)go.transform;
rt.sizeDelta = new Vector2(w, h);
rt.anchoredPosition = pos;
var img = go.GetComponent<Image>();
img.color = new Color(0.16f, 0.24f, 0.32f, 1f);
var btn = go.GetComponent<Button>();
// One-Graphic-per-GO engine law: Tuanjie 2022.3 refuses a second
// Graphic on the same GameObject (AddComponent<Text> returns null,
// X1192 wave-1 proof), so the label rides a stretch-to-parent
// child GO instead.
var labelGo = new GameObject("Label", typeof(RectTransform), typeof(Text));
labelGo.transform.SetParent(go.transform, false);
var lrt = (RectTransform)labelGo.transform;
lrt.anchorMin = Vector2.zero;
lrt.anchorMax = Vector2.one;
lrt.offsetMin = Vector2.zero;
lrt.offsetMax = Vector2.zero;
var label = labelGo.GetComponent<Text>();
label.font = UIFont.Resolve();
label.fontSize = 34;
label.alignment = TextAnchor.MiddleCenter;
label.color = Color.white;
// X1700 AU10: generic tap voice on every programmatic button,
// registered before the host's semantic listener so the beat
// lands after the click (order law, census-neutral new key).
btn.onClick.AddListener(delegate { G16AudioHost.Ensure().PlaySfx("sfx_btn"); });
return btn;
}
// ----- Input -----
void SetDeskOpen(bool open)
{
    _panel.SetActive(open);
    _openButton.gameObject.SetActive(!open);
    if (open) RefreshAlbumFace();
}
void OnOpenTapped()
{
    Taps++;
    ResolveSources();
    CategoryIndex = 0;
    SetDeskOpen(true);
}
void OnTabTapped(int index)
{
    Taps++;
    if (index < 0 || index >= CategoryCount) return;
    CategoryIndex = index;
    RefreshAlbumFace();
}
```

```

    void OnShareCardTapped()
    {
        Taps++;
        ShareTapped(AlbumSim.CardIdShare);
    }
    void OnShareReportTapped()
    {
        Taps++;
        ShareTapped(AlbumSim.CardIdReport);
    }
    void ShareTapped(string cardId)
    {
        string payload;
        if (!TryShare(cardId, _ledger, _stream, _cycle, _telemetry, out payload)) return;
        GUIUtility.systemCopyBuffer = payload; // offline share: the payload IS the product
        LastCardIdShared = cardId;
        // T023-G3: the short-card channel flexes the visual card face;
        // the annual-report channel stays payload-only (long text).
        if (cardId == AlbumSim.CardIdShare) OpenCardFace(payload);
        if (_toastFace != null) _toastFace.text = CopyTable.Get("share_done");
    }
}

// BargainSim.cs
// G16 bargain duel core loop + deal close sim. Pure C#, zero-scene, zero RNG (deterministic Mix32).
// Numeric anchors (numeric pack section 2 / requirements doc M02-M03):
// bubble window 2.5s, max 3 bubbles per duel, fail = price recovers 5% of list (zero-punish),
// down payment 30%, first feedback &lt;= 1.0s ceiling, deal ceremony simultaneity &lt;= 0.8s.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.CrazyEstate
{
    public enum BargainBubbleResult { None, Correct, Wrong, Timeout }
    public enum BargainOptionKind { Pitch, Counter, Decoy }
    public struct BargainOption
    {
        public int TalkId;
        public string CopyKey;
        public BargainOptionKind Kind;
        public bool Highlighted; // guided state: the correct counter shines (U100 SP01 fix)
    }
    public struct BargainDealReceipt
    {
        public bool Accepted;
        public int PriceWan;
        public int DownPayWan;
        public int SavingsWan;
        public string RefusalKey; // empty when accepted
    }
}

```

```

}
public sealed class BargainSim
{
    public const float BubbleWindowSec = 2.5f;
    public const int MaxBubbles = 3;
    public const float FailRecoverPct = 0.05f;
    public const float DownPayPct = 0.30f;
    public const float FirstFeedbackCeilingSec = 1.0f;
    public const float DealCeremonyCeilingSec = 0.8f;
    public int ListPriceWan { get; private set; }
    public int CurrentPriceWan { get; private set; }
    public int BubbleIndex { get; private set; }
    public bool SessionOpen { get; private set; }
    public bool FirstBargainEver { get; private set; }
    public bool DealSigned { get; private set; }
    public int OwnedHouses { get; private set; }
    // Best single-deal cut (wan) - the Bargain God honor reads this.
    public int BestSavingsWan { get; private set; }
    // Redline-11 anchor clock (session seconds, driven by DriveTick).
    public float SessionSec { get; private set; }
    public float FirstPitchAtSec { get; private set; }
    public float FirstFeedbackSec { get; private set; } // elapsed from first pitch
    public float BubbleShownAtSec { get; private set; }
    public float DealSignedAtSec { get; private set; }
    public int CorrectTalkId { get; private set; }
    public BargainBubbleResult LastResult { get; private set; }
    public int CorrectCount { get; private set; }
    public int FailCount { get; private set; }
    // T022-d1a story-hub cut-window factor: the host composes the ring-2
    // ch1 gift (+pct) and the ring-3 Hike dive-price comedy (xN) into
    // ONE factor and sets it per duel; the sim only multiplies. The
    // window only ever WIDENS - anything below the bare 1.0 clamps back
    // to identity, so the T013 bare baseline is unreachable from below
    // (a gain is comfort, never a gate).
    public double CutWindowFactor { get; private set; }
    public void SetCutWindowFactor(double factor)
    {
        CutWindowFactor = factor < 1.0 ? 1.0 : factor;
    }
    public event Action<BargainBubbleResult, int, int> OnBubbleResolved; // result, id, priceAfter
    public event Action<BargainDealReceipt> OnDealSigned;
    public bool IsGuided { get { return FirstBargainEver && BubbleIndex == 0 && CorrectTalkId == Fir
    public const int FirstGuidedTalkId = 1;
    public BargainSim(int ownedHouses = 0)
    {
        OwnedHouses = ownedHouses;
        CutWindowFactor = 1.0;
    }
    public void StartBargain(int listPriceWan, bool firstBargainEver)
    {

```

```

_iaaDoubleButton = MakeButton(_panel.transform, "IaaDoubleButton", new Vector2(-150f, -30f), 300, 96);
_iaaDoubleButton.GetComponent<Image>().color = new Color(0.55f, 0.42f, 0.12f, 1f);
_iaaDoubleButton.GetComponentInChildren<Text>().fontSize = 22;
_iaaDoubleButton.GetComponentInChildren<Text>().text = CopyTable.Get("iaa_double_offer");
_iaaDoubleButton.onClick.AddListener(OnIaaDoubleTapped);
_iaaSkipButton = MakeButton(_panel.transform, "IaaSkipButton", new Vector2(150f, -30f), 300, 96);
_iaaSkipButton.GetComponentInChildren<Text>().fontSize = 22;
_iaaSkipButton.GetComponentInChildren<Text>().text = CopyTable.Get("iaa_refuse");
_iaaSkipButton.onClick.AddListener(OnIaaSkipTapped);
_iaaToast = MakeText(_panel.transform, "IaaToast", 24f, new Vector2(0f, -140f), 620, 46);
_iaaToast.text = string.Empty;
// T020-b2 LIC02 ticket faces: the ring badge + the storage-empire
// aggregation line, both gated on the rent-tycoon seat (LicenseGate
// law, LI15) and boot-hidden so the redline-11 timeline holds.
_holdingsBadge = MakeText(_panel.transform, "HoldingsBadge", 22f, new Vector2(0f, -300f), 300, 44);
_holdingsBadge.color = new Color(0.95f, 0.83f, 0.45f, 1f);
_holdingsBadge.gameObject.SetActive(false);
_storageFace = MakeText(_panel.transform, "StorageFace", 26f, new Vector2(0f, -370f), 620, 64);
_storageFace.gameObject.SetActive(false);
// T008 slice-B: bank-grace crisis popup (T012 preview) - a
// canvas-root child so the bank call shows with the desk closed.
_crisisPopup = new GameObject("CrisisPopup", typeof(RectTransform), typeof(Image));
_crisisPopup.transform.SetParent(canvasGo.transform, false);
var crt = (RectTransform)_crisisPopup.transform;
crt.sizeDelta = new Vector2(640, 420);
crt.anchoredPosition = new Vector2(0f, 0f);
var cimg = _crisisPopup.GetComponent<Image>();
cimg.color = new Color(0.30f, 0.08f, 0.08f, 0.98f);
_crisisFace = MakeText(_crisisPopup.transform, "CrisisFace", 30f, new Vector2(0f, 110f), 600, 120);
_crisisFace.text = CopyTable.Get("iaa_rescue_offer");
_crisisAcceptButton = MakeButton(_crisisPopup.transform, "CrisisAcceptButton",
    new Vector2(-140f, -120f), 280, 96);
_crisisAcceptButton.GetComponent<Image>().color = new Color(0.55f, 0.42f, 0.12f, 1f);
_crisisAcceptButton.GetComponentInChildren<Text>().fontSize = 26;
_crisisAcceptButton.GetComponentInChildren<Text>().text = CopyTable.Get("iaa_accept");
_crisisAcceptButton.onClick.AddListener(OnCrisisAcceptTapped);
var crisisRefuse = MakeButton(_crisisPopup.transform, "CrisisRefuseButton",
    new Vector2(140f, -120f), 280, 96);
crisisRefuse.GetComponentInChildren<Text>().fontSize = 26;
crisisRefuse.GetComponentInChildren<Text>().text = CopyTable.Get("iaa_refuse");
crisisRefuse.onClick.AddListener(OnCrisisRefuseTapped);
_crisisPopup.SetActive(false); // boot hidden: redline-11 timeline law
}

static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;

```

```
        var t = go.GetComponent<Text>();
        t.font = UIFont.Resolve();
        t.fontSize = Mathf.RoundToInt(size);
        t.alignment = TextAnchor.MiddleCenter;
        t.color = Color.white;
        return t;
    }

    static Button MakeButton(Transform parent, string name, Vector2 pos, int w, int h)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = pos;
        var img = go.GetComponent<Image>();
        img.color = new Color(0.18f, 0.22f, 0.30f, 1f);
        var btn = go.GetComponent<Button>();
        // One-Graphic-per-GO engine law: Tuanjie 2022.3 refuses a second
        // Graphic on the same GameObject (AddComponent<Text> returns null,
        // X1192 wave-1 proof), so the label rides a stretch-to-parent
        // child GO instead.
        var labelGo = new GameObject("Label", typeof(RectTransform), typeof(Text));
        labelGo.transform.SetParent(go.transform, false);
        var lrt = (RectTransform)labelGo.transform;
        lrt.anchorMin = Vector2.zero;
        lrt.anchorMax = Vector2.one;
        lrt.offsetMin = Vector2.zero;
        lrt.offsetMax = Vector2.zero;
        var label = labelGo.GetComponent<Text>();
        label.font = UIFont.Resolve();
        label.fontSize = 34;
        label.alignment = TextAnchor.MiddleCenter;
        label.color = Color.white;
        // X1700 AU10: generic tap voice on every programmatic button,
        // registered before the host's semantic listener so the beat
        // lands after the click (order law, census-neutral new key).
        btn.onClick.AddListener(delegate { G16AudioHost.Ensure().PlaySfx("sfx_btn"); });
        return btn;
    }

    // ----- Live refresh + input -----
    void RefreshCollectFace()
    {
        if (_ledger == null || _collectLabel == null) return;
        _collectLabel.text = ComposeCollectLabel(_ledger.PendingCoins);
        // X1690 T023-F3: new pending coins arriving IS the rent rain
        // beat - the audio channel (census class rent_rain) fires on the
        // rise only, never on a plain face repaint.
        if (_ledger.PendingCoins > _lastPendingSeen)
            G16AudioHost.Ensure().PlaySfx("sfx_rent_rain");
        _lastPendingSeen = _ledger.PendingCoins;
    }
}
```

```
// X1277: a face refresh also restores the collect chrome from
// the receipt pulse back to the resting base tint.
if (_collectButton != null)
    _collectButton.GetComponent<Image>().color = CollectBaseTint;
var pairVisible = DoublePairVisible(_dayKey, _declinedDay, _ledger);
_doubleButton.gameObject.SetActive(pairVisible);
_skipButton.gameObject.SetActive(pairVisible);
// T008 slice-B: the ad pair follows the same boot-hidden law with
// its own decline memory (the two pairs are independent offers).
var iaaVisible = IaaDoublePairVisible(AdDayKey(), _iaaDeclinedDay, _dayKey, _iaa, _ledger);
_iaaDoubleButton.gameObject.SetActive(iaaVisible);
_iaaSkipButton.gameObject.SetActive(iaaVisible);
if (_statusText != null)
    _statusText.gameObject.SetActive(_ledger.Rows.Count == 0);
// T020-b2 LIC02 ticket gate: badge + aggregation line read the live
// seat through the desk resolver; visible faces refresh from the
// real ledger (keys + liquid), hidden faces change nothing.
var storageOpen = LicenseGate.StorageFaceVisible(LicenseViewHost.LiveActiveLine());
if (_holdingsBadge != null)
{
    _holdingsBadge.gameObject.SetActive(storageOpen);
    if (storageOpen) _holdingsBadge.text = ComposeHoldingsBadge(_ledger.Rows.Count);
}
if (_storageFace != null)
{
    _storageFace.gameObject.SetActive(storageOpen);
    if (storageOpen) _storageFace.text = ComposeStorageFace(_ledger.Rows.Count, _ledger.LiquidWan);
}
}
void RefreshCrisisFace()
{
    if (_crisisPopup == null || _iaa == null) return;
    var visible = CrisisFaceVisible(AdDayKey(), _crisisDeclinedDay, _dayKey, _iaa);
    if (visible && !_crisisPopup.activeSelf)
    {
        CrisisFaceShown = true;
        // T022-d1c: the bank call going live IS the SYS03 cash-flow
        // tight signal (ring-1 ch9 footprint - the pair the two real
        // sell actions complete).
        StoryRuntime.Feed(GateFeed.CashTight);
        // T012 slice B live trigger face: the distress copy names the
        // two outs - bank-grace ad or sell a house (U091 no dead end).
        _crisisFace.text = CopyTable.Get("crisis_show_title") + "\n"
            + CopyTable.Get("crisis_show_body");
    }
    _crisisPopup.SetActive(visible);
}
void SetDeskOpen(bool open)
{
    _panel.SetActive(open);
}
```

```
        _openButton.gameObject.SetActive(!open);
        if (open) RefreshCollectFace();
    }
    void OnOpenTapped()
    {
        Taps++;
        SetDeskOpen(true);
    }
    // Rent rain: drain the pending pool to lifetime, fold the popup.
    void OnCollectTapped()
    {
        Taps++;
        CollectTaps++;
        LastCollectAmount = _ledger.Collect();
        // X1690 T023-F3: a real drain (coins actually moved) carries the
        // audio channel (census class collect); an empty tap stays silent.
        if (LastCollectAmount > 0)
            G16AudioHost.Ensure().PlaySfx("sfx_collect");
        if (_summaryPopup != null && _summaryPopup.activeSelf) _summaryPopup.SetActive(false);
        RefreshCollectFace();
        // X1277 U213 execution anchor: the drained amount stamps the
        // collect face and pulses the accent until the next face refresh.
        var receipt = ComposeCollectReceipt(LastCollectAmount);
        if (receipt.Length > 0 && _collectLabel != null && _collectButton != null)
        {
            _collectLabel.text = receipt;
            _collectButton.GetComponent<Image>().color = G16StyleAnchor.AccentColor;
        }
    }
    void OnDoubleTapped()
    {
        Taps++;
        DoubleTaps++;
        if (!TryAcceptDoubleTap(_dayKey, _ledger)) return;
        // X1698 AU09: the doubled rain claim carries its own reward
        // sound; the pending seen-marker is bumped first so the
        // rain-rise edge stays quiet for this refresh (no double fire).
        _lastPendingSeen = _ledger.PendingCoins;
        G16AudioHost.Ensure().PlaySfx("sfx_rent_double");
        RefreshCollectFace();
    }
    void OnSkipTapped()
    {
        Taps++;
        SkipTaps++;
        _declinedDay = _dayKey; // offer declined for the rest of today
        RefreshCollectFace();
    }
    // T008 slice-B: ad-gated double - the grant doubles the pending rain
    // through the ledger law and banks one lottery sign; the toast shows
```

```
// the doubled gain (iaa_double_done face).
void OnIaaDoubleTapped()
{
    Taps++;
    IaaDoubleTaps++;
    if (!TryIaaDoubleTap(AdDayKey(), _dayKey, _iaa, _ledger)) return;
    // X1698 AU09: the ad-gated double claim rides the same reward
    // seat (seen-marker bump keeps the rain-rise edge silent here).
    _lastPendingSeen = _ledger.PendingCoins;
    G16AudioHost.Ensure().PlaySfx("sfx_rent_double");
    if (_iaaToast != null)
        _iaaToast.text = ComposeIaaDoubleDone(_ledger.PendingCoins);
    RefreshCollectFace();
}
void OnIaaSkipTapped()
{
    Taps++;
    IaaSkipTaps++;
    _iaaDeclinedDay = _dayKey; // ad pair declined for the rest of today
    RefreshCollectFace();
}
// Bank-grace rescue: the ad clears the distress flag and opens the
// 30-minute grace window (T012 passes a real mortgage id later; S1
// live play never raises the flag, so 0 is a dormant placeholder).
void OnCrisisAcceptTapped()
{
    Taps++;
    CrisisTaps++;
    // T012 slice B: the real distressed row id replaces the dormant 0
    // (0 stays when the ledger is clean - S1 live play never redlines).
    if (_mortgageDesk == null) _mortgageDesk = FindObjectOfType<MortgageViewHost>();
    var mortgageId = _mortgageDesk != null
        ? MortgageViewHost.DistressedUnitId(_mortgageDesk.Sim) : 0;
    if (!TryRescueTap(AdDayKey(), _iaa, mortgageId, Time.time)) return;
    if (_crisisFace != null) _crisisFace.text = CopyTable.Get("iaa_rescue_done");
}
void OnCrisisRefuseTapped()
{
    Taps++;
    CrisisRefuses++;
    _crisisDeclinedDay = _dayKey; // bank call dismissed for today
}
// Welcome-back face: the boot settle report lands here and the desk
// opens itself so the revisit summary is the first thing seen.
public void ShowWelcomeBack(OfflineReport report)
{
    if (_summaryPopup == null) return;
    _summaryBody.text = _ledger.OfflineSummaryText(report);
    _summaryPopup.SetActive(true);
    PopupShown = true;
}
```

```
        SetDeskOpen(true);
    }
}

// TenantTable.cs
// G16 tenant roster (SYS05 RentRow tenant face). Requirements doc tenant table:
// 10 comedy tenants from the Oasis City roster; deterministic per-row
// assignment (zero RNG API), all visible copy via CopyTable EN/ZH faces.
using System.Collections.Generic;
namespace Sy.CrazyEstate
{
    public struct TenantDef
    {
        public int Id;
        public string NameKey;
        public string ReviewKey;
    }

    public static class TenantTable
    {
        public const int Count = 10;
        public static readonly TenantDef[] Tenants =
        {
            new TenantDef { Id = 1, NameKey = "tenant_name_01", ReviewKey = "tenant_review_01" },
            new TenantDef { Id = 2, NameKey = "tenant_name_02", ReviewKey = "tenant_review_02" },
            new TenantDef { Id = 3, NameKey = "tenant_name_03", ReviewKey = "tenant_review_03" },
            new TenantDef { Id = 4, NameKey = "tenant_name_04", ReviewKey = "tenant_review_04" },
            new TenantDef { Id = 5, NameKey = "tenant_name_05", ReviewKey = "tenant_review_05" },
            new TenantDef { Id = 6, NameKey = "tenant_name_06", ReviewKey = "tenant_review_06" },
            new TenantDef { Id = 7, NameKey = "tenant_name_07", ReviewKey = "tenant_review_07" },
            new TenantDef { Id = 8, NameKey = "tenant_name_08", ReviewKey = "tenant_review_08" },
            new TenantDef { Id = 9, NameKey = "tenant_name_09", ReviewKey = "tenant_review_09" },
            new TenantDef { Id = 10, NameKey = "tenant_name_10", ReviewKey = "tenant_review_10" },
        };

        public static TenantDef GetTenant(int id)
        {
            for (var i = 0; i < Tenants.Length; i++)
                if (Tenants[i].Id == id) return Tenants[i];
            return Tenants[0];
        }

        // Deterministic tenant for a portfolio row: pure function of the listing
        // id and the row ordinal (same buys -> same tenants on double run; the
        // 7/11 stride keeps consecutive units from colliding on one tenant).
        public static int TenantForRow(int rowOrdinal, int unitId)
        {
            return ((unitId * 7 + rowOrdinal * 11) % Count) + 1;
        }

        public static List<string> Validate(List<string> problems)
        {
            if (problems == null) problems = new List<string>();
            if (Tenants.Length != Count) problems.Add("tenant roster short of 10");
        }
    }
}
```

```

        var zh = CopyTableZh.Snapshot();
        for (var i = 0; i < Tenants.Length; i++)
        {
            var t = Tenants[i];
            if (!CopyTable.TryGetRaw(t.NameKey, out var n) || string.IsNullOrEmpty(n))
                problems.Add("tenant name key missing: " + t.Id);
            if (!CopyTable.TryGetRaw(t.ReviewKey, out var r) || string.IsNullOrEmpty(r))
                problems.Add("tenant review key missing: " + t.Id);
            if (!zh.ContainsKey(t.NameKey)) problems.Add("tenant zh name missing: " + t.Id);
            if (!zh.ContainsKey(t.ReviewKey)) problems.Add("tenant zh review missing: " + t.Id);
            for (var j = i + 1; j < Tenants.Length; j++)
                if (Tenants[j].Id == t.Id) problems.Add("tenant id duplicated: " + t.Id);
        }
        return problems;
    }
}

// ArcState.cs
// G16 ArcState ring-3 rival-pool arc-state layer (T022-c, M15 Oasis Story
// Almanac section 6 ring 3): gameplay rewrites the story. The six rival-arc
// flip rows of the almanac table ride the ring-1 PlayGate chapter walk and
// rewrite the middleman duel pool - Hike hangs his listings at a dive price
// with the x1.5 comedy cut window after ch7, then leaves the pool for the
// noodle-shop egg seat after ch11 (meeting him again is an egg, not a duel);
// Iron-mouth leaves for the BBQ stall at the ch8 window and returns at the
// ch12 window as the honest agent (small cut, zero trap talk); the lottery
// koi sister enters the pool at the act-3 ch7 window as the rookie seat
// Lucky Hand Jie; Lisa mounts her mentor word-of-mouth star after ch9. Each
// flip fires evt_arc_flip exactly once (table order, idempotent latch). The
// story read-data faces keep the copy honest: the ch11 "peer" verdict reads
// the REAL accumulated Hike duel count, the grandma-livestream viewer
// number reads the REAL total closed-deal count, the ch10 revisit roster
// reads the real TenantTable, and the tycoon annual report reuses SYS09 (no
// duplicate face here). Pure decision layer like PlayGate/DramaBuff: zero
// RNG, zero clocks, culture-invariant fingerprint; host wiring (duel feeds,
// dive-price listings, view consumption) lands in the host slice.
using System;
using System.Globalization;
using System.Text;
namespace Sy.CrazyEstate
{
    public sealed class ArcState
    {
        // Almanac section 6 ring-3 flip table (six rows, verbatim order).
        public const int FlipCount = 6;
        public const int FlipHikeDive = 0;           // Hike ch7: dive price + x1.5 cut window
        public const int FlipHikeNoodleExit = 1;     // Hike ch11+: pool exit -> noodle egg
        public const int FlipIronBbqExit = 2;        // Iron ch8 window: pool exit (BBQ stall)
        public const int FlipIronHonestReturn = 3;    // Iron ch12 window: re-entry, honest rival
        public const int FlipKoiRookieEnter = 4;     // Koi act-3 ch7 window: rookie seat enters
    }
}

```

```

public const int FlipLisaMentorStar = 5; // Lisa ch9+: mentor word-of-mouth star
public const string ArcFlipEventId = "evt_arc_flip";
// Pack-frozen initial anchors (numeric pack section 10.2; the pack
// owns the re-anchor after the bot sweep - no mid-round value edits).
public const double HikeDiveCutFactor = 1.5; // comedy dive-price cut window
public const int HikeDuelPeerAnchor = 3; // ch11 "you treated me as a peer" verdict
public const int LisaMentorStarAdd = 1; // mentor word-of-mouth star
// Codex agent seats (CodexRosterTable): 1 Iron, 2 Weepy, 3 Hike,
// 4 Zen, 5 Lucky Hand Jie (rookie seat, ring-3 entry).
public const int AgentIron = 1;
public const int AgentWeepy = 2;
public const int AgentHike = 3;
public const int AgentZen = 4;
public const int AgentKoi = 5;
// Flip trigger chapters (almanac ring-3 table verbatim).
static readonly int[] FlipChapter = { 7, 11, 8, 12, 7, 9 };
readonly bool[] _flipped = new bool[FlipCount];
int _flipFires;
int _lastFlipOrdinalThisSync;
int _hikeDuels;
int _dealsClosed;
public int FlipFires { get { return _flipFires; } }
public int LastFlipOrdinalThisSync { get { return _lastFlipOrdinalThisSync; } }
public int HikeDuelCount { get { return _hikeDuels; } }
public int TotalDealsClosed { get { return _dealsClosed; } }
public bool IsFlipped(int flip) { return flip >= 0 && flip < FlipCount && _flipped[flip]; }
/// <summary>evt_arc_flip anchor: fires exactly once per flip row,
/// carrying the flip ordinal (the funnel reads which story arcs the
/// gameplay has already rewritten).</summary>
public event Action<int> OnArcFlip;
/// <summary>Apply every ring-3 flip whose trigger chapter the ring-1
/// gate has unlocked (table order, idempotent latch - re-syncing
/// re-fires nothing). Returns how many flips THIS sync applied
/// (0 = nothing new; a zero-chapter player keeps the bare pool).</summary>
public int SyncFromGate(PlayGate gate)
{
    _lastFlipOrdinalThisSync = 0;
    var applied = 0;
    var unlocked = gate.UnlockedChapter;
    for (var f = 0; f < FlipCount; f++)
    {
        if (_flipped[f] || FlipChapter[f] > unlocked) continue;
        _flipped[f] = true;
        applied++;
        _lastFlipOrdinalThisSync = f + 1;
        _flipFires++;
        OnArcFlip?.Invoke(f);
    }
    return applied;
}

```

```
// ---- rival-pool query faces (identity until the flip lands) ----
/// &summary&Duel-pool membership: the rotation draws only from the
/// live pool. Iron is in until the ch8 BBQ exit and returns at the
/// ch12 honest window; Hike is in until his ch11 noodle exit; the
/// koi rookie seat is out until her ch7 window; Weepy and Zen never
/// flip (pool evergreen seats).&summary&
public bool IsAgentInPool(int agentId)
{
    switch (agentId)
    {
        case AgentIron:
            return !_flipped[FlipIronBbqExit] || _flipped[FlipIronHonestReturn];
        case AgentWeepy:
        case AgentZen:
            return true;
        case AgentHike:
            return !_flipped[FlipHikeNoodleExit];
        case AgentKoi:
            return _flipped[FlipKoiRookieEnter];
        default:
            return false;
    }
}

/// &summary&Hike's dive-price comedy egg: after ch7 his listings hang
/// a dive price and the cut window widens x1.5 (misery as comedy) -
/// only while he is still dueling in the pool.&summary&
public bool HikeDivePricingActive()
{
    return _flipped[FlipHikeDive] && !_flipped[FlipHikeNoodleExit];
}

public double HikeCutWindowFactor()
{
    return HikeDivePricingActive() ? HikeDiveCutFactor : 1.0;
}

/// &summary&Meeting Hike after his ch11 exit is the noodle-shop egg,
/// never a duel (his pool seat is gone for good).&summary&
public bool HikeNoodleEggActive() { return _flipped[FlipHikeNoodleExit]; }

/// &summary&Iron's honest re-entry state (ch12 window): small cut,
/// zero trap talk - the agent who learned to tell the truth.&summary&
public bool IronMouthHonestActive() { return _flipped[FlipIronHonestReturn]; }

/// &summary&Lisa's mentor word-of-mouth star (ch9+ agent-line gain,
/// the sweeping-monk cameo promise cashed - mirrors the DramaBuff
/// ch9 star face on the license line).&summary&
public int LisaMentorStarBonus() { return _flipped[FlipLisaMentorStar] ? LisaMentorStarAdd : 0; }

// ---- story read-data faces (copy variables read REAL save values;
// the annual-report face reuses SYS09 - no duplicate face here) ----
/// &summary&Host feeds one closed deal (evt_deal_close): the grandma
/// livestream viewer number reads this REAL total - views are earned,
/// never invented.&summary&
public void FeedDealClosed() { _dealsClosed++; }
```

```

    /// &lt;summary&gt;Host feeds one Hike duel: the ch11 "only you treated me
    /// as a peer" verdict reads this REAL accumulated count (the more you
    /// haggled, the warmer the noodle bowl).&lt;/summary&gt;
    public void FeedHikeDuel() { _hikeDuels++; }
    public bool HikeDuelPeerVerdictMet() { return _hikeDuels &gt;= HikeDuelPeerAnchor; }
    /// &lt;summary&gt;Ch10 revisit-roster window: the roster itself reads the
    /// REAL TenantTable at the view slice (whoever complains the most
    /// gets the biggest comeback scene) - this face only gates the
    /// chapter window, the read-data law keeps TenantTable the single
    /// legal source.&lt;/summary&gt;
    public bool RevisitRosterReady(PlayGate gate) { return gate.IsUnlocked(10); }
    public string Fingerprint()
    {
        var ci = CultureInfo.InvariantCulture;
        var sb = new StringBuilder();
        sb.Append("F");
        for (var f = 0; f &lt; FlipCount; f++) sb.Append(_flipped[f] ? 1 : 0);
        sb.Append(":f").Append(_flipFires);
        sb.Append(":d").Append(_hikeDuels);
        sb.Append(":c").Append(_dealsClosed);
        sb.Append(":p");
        for (var a = 1; a &lt;= AgentKoi; a++) sb.Append(IsAgentInPool(a) ? 1 : 0);
        sb.Append(":w").Append(HikeCutWindowFactor().ToString("0.##", ci));
        sb.Append(":n").Append(HikeNoodleEggActive() ? 1 : 0);
        sb.Append(":h").Append(IronMouthHonestActive() ? 1 : 0);
        sb.Append(":m").Append(LisaMentorStarBonus());
        return sb.ToString();
    }
}

// ArcTheater.cs
// G16 rival-arc act engine + warm-chain late-night theater (T021-b, M15
// Oasis Story Almanac sections 2 and 4). The four rival NPCs (Raise-Bro the
// overnight price-hiker, the Spec-Squad Commander, Top-Seller Lisa, Grandma)
// mount one act line on every story chapter (the twelve chapters quarter into
// four acts), and the warm-chain mini-theater plays its two-beat structure
// (absurd setup -&gt; warm landing) one beat per face. Pure decision layer, zero
// RNG, culture-invariant fingerprints for gate double runs. Redline-11 law
// faces shared with SkylineShow: never play inside the opening window, never
// preempt a running face, every face strictly shorter than the event-popup
// auto-hide (no stacked faces).
using System.Globalization;
using System.Text;
namespace Sy.CrazyEstate
{
    public enum ArcPhase { Idle = 0, Rise = 1, Hold = 2, Fall = 3 }
    public sealed class ArcShow
    {
        // Rival cast (M15 section 2).
        public const int Hike = 0;
    }
}

```

```
public const int Squad = 1;
public const int Lisa = 2;
public const int Granny = 3;
public static readonly int[] RivalIds = { Hike, Squad, Lisa, Granny };
public const float RiseSec = 0.6f;
public const float HoldSec = 2.4f; // the one readable beat
public const float FallSec = 0.8f;
public const float TotalSec = RiseSec + HoldSec + FallSec;
public const float OpeningFaceSec = 10f; // redline-11 opening window anchor
public const int ChapterCount = 12;
// Story quarters: twelve chapters -> four acts (act = chapter quarter).
public static int ActForChapter(int chapter)
{
    if (chapter < 1 || chapter > ChapterCount) return 0;
    return (chapter - 1) / 3 + 1;
}
// Act lines each rival carries (section 2 scripts: Squad joins in
// act 2, Lisa skips act 2 - the others span all four acts).
public static int ActsFor(int rival)
{
    return rival == Hike || rival == Granny ? 4 : 3;
}
// The act a rival mounts on a chapter (0 = sits this quarter out).
public static int ActMountedOn(int rival, int chapter)
{
    var act = ActForChapter(chapter);
    if (act == 0 || rival < 0 || rival > 3) return 0;
    if (rival == Hike || rival == Granny) return act;
    if (rival == Squad) return act == 1 ? 0 : act - 1;
    return act == 1 ? 1 : act == 2 ? 0 : act - 1;
}
public static string KeyFor(int rival, int act)
{
    var name = rival == Hike ? "hike" : rival == Squad ? "squad"
        : rival == Lisa ? "lisa" : "granny";
    return "arc_" + name + "_" + act;
}
ArcPhase _phase;
float _phaseSec;
string _key;
public ArcPhase Phase { get { return _phase; } }
public bool Playing { get { return _phase != ArcPhase.Idle; } }
public string Key { get { return _key; } }
// One act line per face: outside the opening window, never
// preempting, and the copy key must resolve on the table.
public bool BeginAct(int rival, int chapter, float hostSec)
{
    var act = ActMountedOn(rival, chapter);
    if (Playing || act == 0) return false;
    if (hostSec < OpeningFaceSec) return false;
```

```
        if (!CopyTable.TryGetRaw(KeyFor(rival, act), out var line) || string.IsNullOrEmpty(line)) return false;
        _key = KeyFor(rival, act);
        _phase = ArcPhase.Rise;
        _phaseSec = 0f;
        return true;
    }
    public ArcPhase Tick(float dt)
    {
        if (_phase == ArcPhase.Idle || dt <= 0f) return _phase;
        _phaseSec += dt;
        var span = PhaseSpan(_phase);
        if (_phaseSec >= span)
        {
            _phaseSec -= span;
            _phase = _phase == ArcPhase.Rise ? ArcPhase.Hold
                : _phase == ArcPhase.Hold ? ArcPhase.Fall
                : ArcPhase.Idle;
            if (_phase == ArcPhase.Idle) { _phaseSec = 0f; _key = null; }
        }
        return _phase;
    }
    static float PhaseSpan(ArcPhase phase)
    {
        return phase == ArcPhase.Rise ? RiseSec : phase == ArcPhase.Hold ? HoldSec : FallSec;
    }
    public static bool FaceShorterThanPopupAutoHide()
    {
        return TotalSec < EventPopupHost.AutoHideSec;
    }
    public string Fingerprint()
    {
        var sb = new StringBuilder();
        sb.Append("a").Append("p").Append((int)_phase);
        sb.Append(":t").Append(((int)(_phaseSec * 1000)).ToString("D6", CultureInfo.InvariantCulture));
        sb.Append(":k").Append(_key == null ? "-" : _key);
        return sb.ToString();
    }
}
public sealed class TheaterShow
{
    // Warm chains (M15 section 4 late-night mini-theater): W01 the old
    // locust tree (chapters 3 and 12), W02 the south-facing room (ch 9),
    // W03 one bowl between rivals (ch 11).
    public const int W01 = 1;
    public const int W02 = 2;
    public const int W03 = 3;
    public const int BeatsPerChain = 2; // absurd setup -> warm landing
    public const float RiseSec = 0.6f;
    public const float HoldSec = 2.4f;
    public const float FallSec = 0.8f;
```

```
public const float TotalBeatSec = RiseSec + HoldSec + FallSec;
public const float OpeningFaceSec = 10f;
// The chain that mounts on a chapter (section 4 event hooks; 0 = none).
public static int ChainForChapter(int chapter)
{
    return chapter == 3 || chapter == 12 ? W01
        : chapter == 9 ? W02
        : chapter == 11 ? W03
        : 0;
}
public static string TitleKeyFor(int chain)
{
    return "w_theater_w0" + chain + "_title";
}
public static string BeatKeyFor(int chain, int beat)
{
    return "w_theater_w0" + chain + "_beat" + beat;
}
ArcPhase _phase;
float _phaseSec;
int _chain;
int _beat;
public ArcPhase Phase { get { return _phase; } }
public bool Playing { get { return _phase != ArcPhase.Idle; } }
public int Chain { get { return _chain; } }
public int Beat { get { return _beat; } }
// One beat per face: the view starts the next beat only after the
// current face ends (no stacked faces). Same redline-11 guards.
public bool BeginBeat(int chain, int beat, float hostSec)
{
    if (Playing) return false;
    if (chain < W01 || chain > W03 || beat < 1 || beat > BeatsPerChain) return false;
    if (hostSec < OpeningFaceSec) return false;
    if (!CopyTable.TryGetRaw(BeatKeyFor(chain, beat), out var line) || string.IsNullOrEmpty(line)) return false;
    _chain = chain;
    _beat = beat;
    _phase = ArcPhase.Rise;
    _phaseSec = 0f;
    return true;
}
public ArcPhase Tick(float dt)
{
    if (_phase == ArcPhase.Idle || dt <= 0f) return _phase;
    _phaseSec += dt;
    var span = _phase == ArcPhase.Rise ? RiseSec : _phase == ArcPhase.Hold ? HoldSec : FallSec;
    if (_phaseSec >= span)
    {
        _phaseSec -= span;
        _phase = _phase == ArcPhase.Rise ? ArcPhase.Hold
            : _phase == ArcPhase.Hold ? ArcPhase.Fall
```

```

        : ArcPhase.Idle;
        if (_phase == ArcPhase.Idle) { _phaseSec = 0f; _chain = 0; _beat = 0; }
    }
    return _phase;
}
public static bool BeatShorterThanPopupAutoHide()
{
    return TotalBeatSec < EventPopupHost.AutoHideSec;
}
public string Fingerprint()
{
    var sb = new StringBuilder();
    sb.Append("w").Append("p").Append((int)_phase);
    sb.Append(":t").Append(((int)(_phaseSec * 1000)).ToString("D6", CultureInfo.InvariantCulture));
    sb.Append(":c").Append(_chain).Append(":b").Append(_beat);
    return sb.ToString();
}
}
}
// DramaBuff.cs
// G16 DramaBuff ring-2 gain chain (T022-b, M15 Oasis Story Almanac section
// 6): every chapter end mounts one flavor gain the story cast gifts to the
// gameplay (ch1 grandma's video check-in -&gt; bargain window +10%, ch2 the
// landlady coaches rent collection -&gt; first-month rent +20%, ..., ch11 the
// noodle bowl -&gt; every agent talk cut +1 wan, ch12 the city crown -&gt; finale
// halo). The chain is STRICTLY OPTIONAL comfort (the gain law): a player who
// never advances a chapter keeps the full bare baseline on every face - the
// T013 bot sweep proves the no-buff baseline passes, a gain is never a gate
// (U091 zero-harm). Mounts ride the ring-1 PlayGate chapter walk (one gain
// per chapter, strict linear order) and fire evt_drama_buff exactly once per
// gain (the funnel reads "players chase the story for the gains"). Pure
// decision layer like PlayGate: zero RNG, zero clocks, culture-invariant
// fingerprint; effect consumers wire in the host slice - this layer owns the
// mount state and the query faces only.
using System;
using System.Globalization;
using System.Text;
namespace Sy.CrazyEstate
{
    public sealed class DramaBuff
    {
        public const int GainCount = 12;
        public const string GainMountEventId = "evt_drama_buff";
        // Pack-frozen initial anchors (numeric pack section 10.1; the pack
        // owns the re-anchor after the bot sweep - no mid-round value edits).
        public const double BargainWindowPct = 10.0;    // ch1 grandma checks in
        public const double FirstMonthRentPct = 20.0;   // ch2 rent coaching
        public const double DemolitionWeightAdd = 1.0;  // ch3 tea-party intel
        public const int LotteryToleranceBeats = 1;     // ch4 koi mantra
        public const double LicenseGiftPct = 3.0;       // ch5 squad joining gift (flat first freeze)
    }
}

```

```
public const double MortgageDecayAdd = 0.05;      // ch7 the koi's milk tea
public const double RevivalSpeedFactor = 1.5;     // ch8 lamp-keeper mutual aid
public const double ShareExposureAdd = 1.0;       // ch9 grandma livestream debut
public const int ReputationStarAdd = 1;           // ch9 word-of-mouth star
public const double RentRainQuotaAdd = 1.0;       // ch10 tenant revisit squad
public const double AgentCutBonusWan = 1.0;       // ch11 the noodle bowl
public const double FinalHaloPct = 2.0;          // ch12 the city crown halo
int _mounted;      // gains mounted 1.._mounted (linear, never skips)
int _mountFires;
int _lastMountThisSync;
public int MountedCount { get { return _mounted; } }
public int MountFires { get { return _mountFires; } }
public int LastMountedThisSync { get { return _lastMountThisSync; } }
public bool IsMounted(int chapter) { return chapter >= 1 && chapter <= _mounted; }
// <summary>evt_drama_buff anchor: fires exactly once per gain,
// carrying the chapter whose story ending gifted it.</summary>
public event Action<int> OnGainMounted;
// <summary>Mount every gain whose chapter the ring-1 gate has
// unlocked. Idempotent latch: re-syncing the same walk re-fires
// nothing. Returns how many gains THIS sync mounted (0 = nothing
// new - the zero-chapter player stays on the bare baseline).</summary>
public int SyncFromGate(PlayGate gate)
{
    _lastMountThisSync = 0;
    var mountedThisSync = 0;
    var target = gate.UnlockedChapter;
    if (target > GainCount) target = GainCount;
    while (_mounted < target)
    {
        _mounted++;
        mountedThisSync++;
        _lastMountThisSync = _mounted;
        _mountFires++;
        OnGainMounted?.Invoke(_mounted);
    }
    return mountedThisSync;
}

// ---- effect query faces: identity until the gifting chapter lands ----
// (every face returns the bare-baseline value when the gain is not
// mounted, so the T013 numerics sweep is structurally passable with
// zero chapter progress - comfort only, never a required gate).
public double BargainWindowBonusPct() { return IsMounted(1) ? BargainWindowPct : 0.0; }
public double FirstMonthRentBonusPct() { return IsMounted(2) ? FirstMonthRentPct : 0.0; }
public double DemolitionEventWeightAdd() { return IsMounted(3) ? DemolitionWeightAdd : 0.0; }
public int LotteryToleranceBonusBeats() { return IsMounted(4) ? LotteryToleranceBeats : 0; }
public double LicenseGiftBonusPct(int licenseTier) { return IsMounted(5) ? LicenseGiftPct : 0.0; }
public bool MarketDownRentFloorActive() { return IsMounted(6); }
public double MortgagePressureDecayAdd() { return IsMounted(7) ? MortgageDecayAdd : 0.0; }
public double RevivalSpeedBonusFactor() { return IsMounted(8) ? RevivalSpeedFactor : 1.0; }
public double ShareExposureBonus() { return IsMounted(9) ? ShareExposureAdd : 0.0; }
```

```

    public int ReputationStarBonus() { return IsMounted(9) ? ReputationStarAdd : 0; }
    public double RentRainQuotaBonus() { return IsMounted(10) ? RentRainQuotaAdd : 0.0; }
    public double AgentTalkCutBonusWan() { return IsMounted(11) ? AgentCutBonusWan : 0.0; }
    public double FinalHaloBonusPct() { return IsMounted(12) ? FinalHaloPct : 0.0; }
    public string Fingerprint()
    {
        var ci = CultureInfo.InvariantCulture;
        var sb = new StringBuilder();
        sb.Append("M").Append(_mounted);
        sb.Append(":f").Append(_mountFires);
        sb.Append(":w").Append(BargainWindowBonusPct().ToString("0.##", ci));
        sb.Append(":r").Append(FirstMonthRentBonusPct().ToString("0.##", ci));
        sb.Append(":d").Append(DemolitionEventWeightAdd().ToString("0.##", ci));
        sb.Append(":l").Append(LotteryToleranceBonusBeats());
        sb.Append(":g").Append(LicenseGiftBonusPct(1).ToString("0.##", ci));
        sb.Append(":mk").Append(MarketDownRentFloorActive() ? 1 : 0);
        sb.Append(":p").Append(MortgagePressureDecayAdd().ToString("0.##", ci));
        sb.Append(":v").Append(RevivalSpeedBonusFactor().ToString("0.##", ci));
        sb.Append(":s").Append(ShareExposureBonus().ToString("0.##", ci)).Append(ReputationStarBonus());
        sb.Append(":q").Append(RentRainQuotaBonus().ToString("0.##", ci));
        sb.Append(":a").Append(AgentTalkCutBonusWan().ToString("0.##", ci));
        sb.Append(":h").Append(FinalHaloBonusPct().ToString("0.##", ci));
        return sb.ToString();
    }
}

// PlayGate.cs
// G16 PlayGate chapter-gate data layer (T022-a ring 1, M15 Oasis Story Almanac
// section 6): the twelve story chapters unlock ONLY on real gameplay milestones
// - never a time wall. The gate latches the player's footprint facts (first
// deal, first reno, first tenant, demolition, lottery, wealth, first pot,
// crisis, H07 holding, cash-tight sells, tenant scale, five-line finale, city
// crown), walks the chapters in strict linear order (an unmet gate blocks every
// later chapter - no skipping), and fires evt_chapter_unlock exactly once per
// chapter. Pure decision layer, zero RNG, culture-invariant fingerprint; the
// View slice consumes UnlockedChapter / LastUnlockedThisFeed to mount the
// ArcShow act seats (X1114 mount points).
using System;
using System.Globalization;
using System.Text;
namespace Sy.CrazyEstate
{
    /// <summary>Gameplay milestone feeds (SYS anchors, almanac section 6 ring-1 table).</summary>
    public enum GateFeed
    {
        DealClosed = 0,          // evt_deal_close: intPayload = listing id
        RenoCompleted = 1,       // SYS04 first renovation completed
        TenantMovedIn = 2,       // SYS05 first tenant moved in
        DemolitionEvent = 3,     // evt_demolition (E02 gold rain)
        LotteryEntered = 4,      // SYS08 first lottery draw entered
    }
}

```

```
NetWorthSnapshot = 5, // asset snapshot: payload = net worth (wan)
ResaleSettled = 6,    // resale settle: payload = single-flip profit (wan)
MortgageCrisis = 7,  // SYS07 first mortgage-crisis pressure event
HoldingH07 = 8,      // SYS01 holding H07 Future-City pre-sale
CashTight = 9,       // SYS03 cash-flow tight signal
HouseSold = 10,      // SYS03 sell action (count feed)
TenantsHoused = 11,  // intPayload = current tenants-housed count
PackageLicense = 12, // LIC02 landlord license unlocked
LinesAtFinale = 13,  // intPayload = five-line finale-tier count (M14)
}
public sealed class PlayGate
{
    public const int ChapterCount = 12;
    // Pack-frozen anchors (G16 numeric pack section 10): the ch5 wealth
    // line re-anchors to the LIC04 family verdict (X-batch bot sweep) -
    // the 400-wan boot grant crossed the old 90-wan anchor on the very
    // first month beat (a boot-grant trivial unlock, the X1153 debt);
    // 600 wan sits one growth milestone above the boot grant. The ch6
    // first-pot anchor is an initial freeze - the bot sweep re-anchors
    // per the standing pack law (no mid-round value edits).
    public const double NetWorthGateWan = 600.0;
    public const double FirstPotProfitWan = 10.0;
    public const int Ch9SellCountGate = 2;
    public const int Ch10TenantsGate = 5;
    public const int Ch11LinesGate = 2;
    // Identity anchors use the ListingTable id space (1-based, H01=1..H10=10
    // - the same ids BargainViewHost.BoundListingId carries into FeedDealClosed).
    public const int CityCrownListingId = 10; // H10 the world crown (ListingTable Id)
    public const int FutureCityListingId = 7;  // H07 the future city (SYS01 pre-sale dream)
    public const string ChapterUnlockEventId = "evt_chapter_unlock";
    // footprint latches: player facts, monotone where numeric
    bool _dealClosed;
    bool _dealClosedCityCrown;
    bool _renoDone;
    bool _tenantIn;
    bool _demolitionSeen;
    bool _lotteryEntered;
    double _netWorthWanMax;
    double _resaleProfitWanMax;
    bool _mortgageCrisisSeen;
    bool _holdingH07;
    bool _cashTight;
    int _sellCount;
    int _tenantsHousedMax;
    bool _packageLicense;
    int _linesAtFinaleMax;
    int _unlocked;
    int _lastUnlockThisFeed;
    int _unlockFires;
    public int UnlockedChapter { get { return _unlocked; } }
```

```
public int LastUnlockedThisFeed { get { return _lastUnlockThisFeed; } }
public int UnlockFires { get { return _unlockFires; } }
public bool IsUnlocked(int chapter) { return chapter >= 1 && chapter <= _unlocked; }
/// <summary>evt_chapter_unlock anchor: fires exactly once per chapter,
/// carrying the unlocked chapter number (the funnel reads "where the
/// player got to = where the story got to").</summary>
public event Action<int> OnChapterUnlocked;
/// <summary>Feed one gameplay milestone (latch update + gate walk). The
/// return mirrors LastUnlockedThisFeed (highest chapter THIS feed
/// unlocked; 0 = the armed gate still waits - zero dead ends, keep
/// playing the core loop).</summary>
public int Feed(GateFeed feed, double payload = 0.0, int intPayload = 0)
{
    _lastUnlockThisFeed = 0;
    switch (feed)
    {
        case GateFeed.DealClosed:
            _dealClosed = true;
            if (intPayload == CityCrownListingId) _dealClosedCityCrown = true;
            break;
        case GateFeed.RenoCompleted: _renoDone = true; break;
        case GateFeed.TenantMovedIn: _tenantIn = true; break;
        case GateFeed.DemolitionEvent: _demolitionSeen = true; break;
        case GateFeed.LotteryEntered: _lotteryEntered = true; break;
        case GateFeed.NetWorthSnapshot:
            if (payload > _netWorthWanMax) _netWorthWanMax = payload;
            break;
        case GateFeed.ResaleSettled:
            if (payload > _resaleProfitWanMax) _resaleProfitWanMax = payload;
            break;
        case GateFeed.MortgageCrisis: _mortgageCrisisSeen = true; break;
        case GateFeed.HoldingH07: _holdingH07 = true; break;
        case GateFeed.CashTight: _cashTight = true; break;
        case GateFeed.HouseSold: _sellCount++; break;
        case GateFeed.TenantsHoused:
            if (intPayload > _tenantsHousedMax) _tenantsHousedMax = intPayload;
            break;
        case GateFeed.PackageLicense: _packageLicense = true; break;
        case GateFeed.LinesAtFinale:
            if (intPayload > _linesAtFinaleMax) _linesAtFinaleMax = intPayload;
            break;
    }
    WalkGates();
    return _lastUnlockThisFeed;
}
/// <summary>Walk the armed gate only: chapters advance strictly in
/// order, each chapter only after every previous gate hit (a deep
/// footprint can cascade several chapters in one feed, but the order
/// never skips).</summary>
void WalkGates()
```

```
{
    while (_unlocked &lt; ChapterCount &amp;&amp; GateMet(_unlocked + 1))
    {
        _unlocked++;
        _lastUnlockThisFeed = _unlocked;
        _unlockFires++;
        OnChapterUnlocked?.Invoke(_unlocked);
    }
}

/// &lt;summary&gt;The unlock condition of a chapter (almanac section 6
/// ring-1 gate table). Every condition is a latched gameplay footprint
/// - the gate never reads a clock, so no time wall can exist.&lt;/summary&gt;
public bool GateMet(int chapter)
{
    switch (chapter)
    {
        case 1: return _dealClosed;
        case 2: return _renoDone &amp;&amp; _tenantIn;
        case 3: return _demolitionSeen;
        case 4: return _lotteryEntered;
        case 5: return _netWorthWanMax &gt;= NetWorthGateWan;
        case 6: return _resaleProfitWanMax &gt;= FirstPotProfitWan;
        case 7: return _mortgageCrisisSeen;
        case 8: return _holdingH07;
        case 9: return _cashTight &amp;&amp; _sellCount &gt;= Ch9SellCountGate;
        case 10: return _tenantsHousedMax &gt;= Ch10TenantsGate || _packageLicense;
        case 11: return _linesAtFinaleMax &gt;= Ch11LinesGate;
        case 12: return _dealClosedCityCrown;
        default: return false;
    }
}

public string Fingerprint()
{
    var sb = new StringBuilder();
    sb.Append("g").Append(_unlocked);
    sb.Append(":f").Append(_unlockFires);
    sb.Append(":d")
        .Append(_dealClosed ? 1 : 0).Append(_dealClosedCityCrown ? 1 : 0)
        .Append(_renoDone ? 1 : 0).Append(_tenantIn ? 1 : 0)
        .Append(_demolitionSeen ? 1 : 0).Append(_lotteryEntered ? 1 : 0)
        .Append(_mortgageCrisisSeen ? 1 : 0).Append(_holdingH07 ? 1 : 0)
        .Append(_cashTight ? 1 : 0).Append(_packageLicense ? 1 : 0);
    sb.Append(":s").Append(_sellCount);
    sb.Append(":n").Append(_netWorthWanMax.ToString("0.##", CultureInfo.InvariantCulture));
    sb.Append(":r").Append(_resaleProfitWanMax.ToString("0.##", CultureInfo.InvariantCulture));
    sb.Append(":t").Append(_tenantsHousedMax);
    sb.Append(":l").Append(_linesAtFinaleMax);
    return sb.ToString();
}
}
```

```
}
// StoryFeed.cs
// G16 StoryFeed read-model (T022-d3 slice a, M15 Oasis Story Almanac
// section 6, R185 feed two-shape law): the footprint-feed READ layer between
// the recorded story event stream and the future View faces - a BitLife-style
// vertical event stream (the story beats the player actually lived, newest on
// top, mapped verbatim from StoryRuntime.Stream: chapter-gate unlocks,
// comfort-gain mounts, rival-pool arc flips) plus the travel-frog-style
// album collection page (twelve chapter cards, the six rare arc-flip
// points, one collection progress line - every total rides the ring
// constants, never a hand-packed number). The model reads recorded history
// ONLY (the stream and the hub mounts) - it never re-derives a footprint
// and never invents one. The View slice mounts these faces at the
// chapter-gate sidebar / story-page timeline (feed position law: never a
// main-interface seat). Pure decision layer like the rings: zero RNG, zero
// clocks, culture-invariant fingerprint.
using System;
using System.Collections.Generic;
using System.Text;
namespace Sy.CrazyEstate
{
    /// <summary>Which ring anchor one feed row came from.</summary>
    public enum StoryFeedRowKind
    {
        ChapterUnlock = 0, // evt_chapter_unlock: Chapter = unlocked chapter
        GainMount = 1,     // evt_drama_buff: Chapter = the gifting chapter
        ArcFlip = 2        // evt_arc_flip: FlipOrdinal = flip ordinal
    }
    /// <summary>One vertical-feed row (BitLife-style stream entry), mapped
    /// verbatim from one StoryEventRecord - the View mounts its faces on
    /// these fields, the model adds no text of its own (faces come with the
    /// View slice).</summary>
    public sealed class StoryFeedRow
    {
        public StoryFeedRowKind Kind;
        public int Seq;           // stream order, 1-based (latest = biggest)
        public int Chapter;       // unlock / gain chapter (0 for arc flips)
        public int FlipOrdinal; // arc-flip ordinal (0 for unlock / gain)
    }
    public static class StoryFeed
    {
        // Collectible totals ride the ring constants - structural counts,
        // not numeric-pack anchors.
        public static int ChapterTotal { get { return PlayGate.ChapterCount; } }
        public static int GainTotal { get { return DramaBuff.GainCount; } }
        public static int RareFlipTotal { get { return ArcState.FlipCount; } }
        /// <summary>Every collectible story moment the album tracks:
        /// chapter unlocks + gain mounts + rare arc flips.</summary>
        public static int MomentTotal
        {

```

```
        get { return ChapterTotal + GainTotal + RareFlipTotal; }
    }
    /// <summary>The vertical-feed rows, NEWEST FIRST (the feed shows the
    /// latest lived beat on top), mapped verbatim from the recorded
    /// stream - history as it happened, never re-derived.</summary>
    public static List<StoryFeedRow> Rows()
    {
        var stream = StoryRuntime.Stream;
        var rows = new List<StoryFeedRow>(stream.Count);
        for (var i = stream.Count - 1; i >= 0; i--) rows.Add(MapRow(stream[i]));
        return rows;
    }
    /// <summary>Feed length (one row per recorded story event).</summary>
    public static int RowCount { get { return StoryRuntime.Stream.Count; } }
    /// ---- album collection page (travel-frog style) ----
    /// <summary>One album card: has this chapter gate been collected
    /// (unlocked) yet? Out-of-range chapters are never collected.</summary>
    public static bool AlbumChapterUnlocked(int chapter)
    {
        return StoryRuntime.Gate.IsUnlocked(chapter);
    }
    /// <summary>Album progress line: chapters collected so far.</summary>
    public static int AlbumUnlockedCount
    {
        get { return StoryRuntime.Gate.UnlockedChapter; }
    }
    /// <summary>Rare points collected: distinct arc flips recorded in
    /// the stream (the single-fire law makes stream count = distinct).</summary>
    public static int RareFlipCount
    {
        get
        {
            var seen = new bool[ArcState.FlipCount];
            var count = 0;
            foreach (var e in StoryRuntime.Stream)
            {
                if (e.EventId != ArcState.ArcFlipEventId) continue;
                if (e.Payload < 0 || e.Payload >= ArcState.FlipCount) continue;
                if (seen[e.Payload]) continue;
                seen[e.Payload] = true;
                count++;
            }
            return count;
        }
    }
    /// <summary>Album collection progress: story moments lived so far
    /// (every stream record is one collected moment).</summary>
    public static int CollectedMoments { get { return RowCount; } }
    /// <summary>Feed-model fingerprint: every row verbatim plus the
    /// album faces - the CH19 double-run determinism face.</summary>
```

```
public static string FeedFingerprint()
{
    var sb = new StringBuilder();
    foreach (var r in Rows())
        sb.Append((int)r.Kind).Append('.').Append(r.Seq).Append('.')
            .Append(r.Chapter).Append('.').Append(r.FlipOrdinal).Append(';');
    sb.Append('|')
        .Append("u").Append(AlbumUnlockedCount)
        .Append("f").Append(RareFlipCount)
        .Append("m").Append(CollectedMoments)
        .Append("t").Append(MomentTotal);
    return sb.ToString();
}
static StoryFeedRow MapRow(StoryEventRecord e)
{
    var row = new StoryFeedRow();
    row.Seq = e.Seq;
    if (e.EventId == PlayGate.ChapterUnlockEventId)
    {
        row.Kind = StoryFeedRowKind.ChapterUnlock;
        row.Chapter = e.Payload;
    }
    else if (e.EventId == DramaBuff.GainMountEventId)
    {
        row.Kind = StoryFeedRowKind.GainMount;
        row.Chapter = e.Payload;
    }
    else
    {
        row.Kind = StoryFeedRowKind.ArcFlip;
        row.FlipOrdinal = e.Payload;
    }
    return row;
}
}
}
// StoryFeedViewHost.cs
// G16 story feed view host (T022-d3 slice b, M15 Oasis Story Almanac
// section 6 / R185 feed two-shape + position law): the READ face of the
// footprint feed - a chapter-gate sidebar strip on the right edge (the
// gate progress face, tap to open) and the story page it opens: the
// Bitlife-style vertical timeline (every recorded row one face line,
// newest on top) plus the travel-frog-style collection album line.
// Position law: the whole face rides ONE canvas at order 2, strictly
// under every wall and desk face (3-11), the popup (12) and the act
// show (13) - an edge/reading seat that can never preempt a gameplay
// face, never a main-interface position; the page opens ONLY from the
// sidebar tab tap (passive read model, the host never feeds events).
// Faces compose from CopyTable through the pure decision layer below
// (single truth, shared with ChapterCheck CH19); live data is pulled
```

```
// from StoryFeed on a slow refresh clock, so no Start-order race can
// show a stale face. Programmatic UGUI, zero scene authoring (wired by
// SceneWire16 W18).
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.CrazyEstate
{
    public sealed class StoryFeedViewHost : MonoBehaviour
    {
        // Feed position law (R185): one canvas strictly under every wall
        // and desk face - the feed is an auxiliary seat, never the main
        // interface. The sidebar tab lives on the right edge of the 720
        // reference frame, away from every centered desk face.
        public const int CanvasOrder = 2;
        // right-edge anchor: the widest sidebar face (GateFace, 100px) ends
        // at 700 of the 720 reference frame - 300 = 360 - 50 - 10px safety
        // inset, so the docked seat never READS as edge-cropped (X1207 R2:
        // the flush 310 edge passed the in-frame law but reviewed as a
        // half-cropped card; was 318 = hard clip, X1192 P18 law)
        public const float SidebarEdgeX = 300f;
        public const float PageWidth = 700f;
        public const float PageHeight = 940f;
        public const float FaceRefreshSec = 0.5f;
        GameObject _panel;
        Button _tabButton;
        Text _gateFace;
        Text _latestFace;
        Text _albumTitleFace;
        Text _albumProgressFace;
        Text _timelineFace;
        Button _closeButton;
        float _faceClock;
        public int Taps;
        public bool PageOpen { get { return _panel != null && _panel.activeSelf; } }
        // ----- Pure decision layer (single truth, shared with ChapterCheck) -----
        // Copy key helpers: chapter titles reuse the T021-a almanac set
        // verbatim, rare flips carry their own almanac names (1-based keys).
        public static string ChapterTitleKey(int chapter)
        {
            return "chapter_title_" + chapter.ToString("00", CultureInfo.InvariantCulture);
        }
        public static string FlipNameKey(int flipOrdinal)
        {
            return "feed_flip_" + (flipOrdinal + 1).ToString("00", CultureInfo.InvariantCulture);
        }
        // Chapter-gate sidebar face: the live gate progress (chapters
        // walked / total, moments lived / total) - honest zeros on the
```

```

// bare run, never invented.
public static string ComposeSidebarFace()
{
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("feed_sidebar_face_fmt"),
        StoryFeed.AlbumUnlockedCount, StoryFeed.ChapterTotal,
        StoryFeed.CollectectedMoments, StoryFeed.MomentTotal);
}
// Sidebar latest-beat preview (U198 content axis, X1240): the
// sidebar carries the newest lived story beat verbatim (newest
// row first per the feed law) - or the waiting hint on the bare
// run. Reads the recorded stream only, never invents a beat.
public static string ComposeLatestFace()
{
    if (StoryFeed.RowCount == 0) return CopyTable.Get("feed_latest_none");
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("feed_latest_fmt"), ComposeRowFace(StoryFeed.Rows()[0]));
}
// Album progress face: chapters collected, rare flips collected,
// moments lived - against the ring-constant totals.
public static string ComposeAlbumProgressFace()
{
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("feed_album_progress_fmt"),
        StoryFeed.AlbumUnlockedCount, StoryFeed.ChapterTotal,
        StoryFeed.RareFlipCount, StoryFeed.RareFlipTotal,
        StoryFeed.CollectectedMoments, StoryFeed.MomentTotal);
}
// One timeline face line per recorded row: unlocks and gains carry
// the chapter number plus the almanac chapter title verbatim, rare
// flips carry the almanac flip name. The face only reads the row -
// it never re-derives or invents story facts.
public static string ComposeRowFace(StoryFeedRow row)
{
    {
        var inv = CultureInfo.InvariantCulture;
        if (row.Kind == StoryFeedRowKind.ArcFlip)
        {
            var name = row.FlipOrdinal >= 0 && row.FlipOrdinal < ArcState.FlipCount
                ? CopyTable.Get(FlipNameKey(row.FlipOrdinal))
                : string.Empty;
            return string.Format(inv, CopyTable.Get("feed_row_flip_fmt"), name);
        }
        var title = row.Chapter >= 1 && row.Chapter <= PlayGate.ChapterCount
            ? CopyTable.Get(ChapterTitleKey(row.Chapter))
            : string.Empty;
        return string.Format(inv,
            CopyTable.Get(row.Kind == StoryFeedRowKind.ChapterUnlock
                ? "feed_row_chapter_fmt" : "feed_row_gain_fmt"),
            row.Chapter.ToString(inv), title);
    }
}

```

```
// The story-page timeline body: one face line per row, newest
// first (StoryFeed.Rows is already newest-first), the empty-feed
// copy when nothing has happened yet.
public static string ComposeTimelineBody(List<StoryFeedRow> rows)
{
    if (rows == null || rows.Count == 0) return CopyTable.Get("feed_empty");
    var sb = new StringBuilder();
    for (var i = 0; i < rows.Count; i++)
    {
        if (i > 0) sb.Append('\n');
        sb.Append(ComposeRowFace(rows[i]));
    }
    return sb.ToString();
}

// Face fingerprint: sidebar + album + full timeline compose - the
// CH19 consumption-face double-run determinism check.
public static string FeedFaceFingerprint()
{
    var sb = new StringBuilder();
    sb.Append("s:").Append(ComposeSidebarFace()).Append(';');
    sb.Append("a:").Append(ComposeAlbumProgressFace()).Append(';');
    sb.Append("t:").Append(ComposeTimelineBody(StoryFeed.Rows())).Append(';');
    sb.Append("l:").Append(ComposeLatestFace());
    return sb.ToString();
}

// ----- Face wiring -----
void Start()
{
    BuildUi();
    SetPageOpen(false);
}

void Update()
{
    _faceClock += Time.deltaTime;
    if (_faceClock < FaceRefreshSec) return;
    _faceClock = 0f;
    if (_gateFace != null) _gateFace.text = ComposeSidebarFace();
    if (_latestFace != null) _latestFace.text = ComposeLatestFace();
    if (PageOpen) RefreshPageFaces();
}

void SetPageOpen(bool open)
{
    if (_panel != null) _panel.SetActive(open);
    if (open) RefreshPageFaces();
}

void RefreshPageFaces()
{
    if (_albumProgressFace != null)
        _albumProgressFace.text = ComposeAlbumProgressFace();
    if (_timelineFace != null)
```

```

        _timelineFace.text = ComposeTimelineBody(StoryFeed.Rows());
    }
    void OnTabTapped()
    {
        Taps++;
        SetPageOpen(true);
    }
    void OnCloseTapped()
    {
        Taps++;
        SetPageOpen(false);
    }
    // ----- UI construction (programmatic, zero scene authoring) -----
    void BuildUi()
    {
        var canvasGo = new GameObject("StoryFeedCanvas", typeof(Canvas), typeof(CanvasScaler),
            typeof(GraphicRaycaster));
        var canvas = canvasGo.GetComponent<Canvas>();
        canvas.renderMode = RenderMode.ScreenSpaceOverlay;
        canvas.sortingOrder = CanvasOrder; // feed position law: under every wall/desk face
        var scaler = canvasGo.GetComponent<CanvasScaler>();
        // U184: ScaleWithScreenSize or the 720x1280 reference frame never
        // drives device scale (raw ConstantPixelSize = shrunken UI on phone).
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        scaler.referenceResolution = new Vector2(720, 1280);
        // U262 safe-area law (17 doc section 4, B641 pattern): every
        // screen rides one full-stretch SafeArea child squeezed to
        // Screen.safeArea (notch/rounded-corner squeeze, zero clip).
        canvasGo = SafeAreaRoot16.Mount(canvasGo);
        // Chapter-gate sidebar: right-edge tab + the live gate face.
        _tabButton = MakeButton(canvasGo.transform, "StoryTab",
            new Vector2(SidebarEdgeX, 60f), 96, 76);
        _tabButton.GetComponentInChildren<Text>().fontSize = 26;
        _tabButton.GetComponentInChildren<Text>().text = CopyTable.Get("feed_sidebar_tab");
        _tabButton.onClick.AddListener(OnTabTapped);
        _gateFace = MakeText(canvasGo.transform, "GateFace", 16f,
            new Vector2(SidebarEdgeX, -18f), 100, 120);
        _gateFace.raycastTarget = false; // passive strip under the tab
        _gateFace.text = ComposeSidebarFace();
        // A5 content fill (U198, X1240): the latest-beat preview strip
        // under the gate face - the sidebar reads live before the page
        // opens. Same right-edge column, wrapped beat lines read
        // top-down, passive (never a tap thief).
        _latestFace = MakeText(canvasGo.transform, "LatestFace", 14f,
            new Vector2(SidebarEdgeX, -160f), 100, 150);
        _latestFace.raycastTarget = false;
        _latestFace.alignment = TextAnchor.UpperLeft;
        _latestFace.text = ComposeLatestFace();
        // Story page (timeline + album), hidden until the tab tap.
        _panel = new GameObject("StoryPage", typeof(RectTransform), typeof(Image));
    }

```

```

_panel.transform.SetParent(canvasGo.transform, false);
var prt = (RectTransform)_panel.transform;
prt.sizeDelta = new Vector2(PageWidth, PageHeight);
prt.anchoredPosition = new Vector2(0f, -60f);
var img = _panel.GetComponent<Image>();
img.color = new Color(0.07f, 0.10f, 0.14f, 0.97f);
var titleFace = MakeText(_panel.transform, "StoryTitle", 40f,
    new Vector2(0f, 420f), 660, 64);
titleFace.text = CopyTable.Get("feed_page_title");
_albumTitleFace = MakeText(_panel.transform, "AlbumTitle", 24f,
    new Vector2(0f, 352f), 660, 44);
_albumTitleFace.text = CopyTable.Get("feed_album_title");
_albumProgressFace = MakeText(_panel.transform, "AlbumProgress", 21f,
    new Vector2(0f, 306f), 660, 42);
_albumProgressFace.raycastTarget = false;
_timelineFace = MakeText(_panel.transform, "TimelineFace", 19f,
    new Vector2(0f, -42f), 640, 690);
_timelineFace.alignment = TextAnchor.UpperLeft;
_timelineFace.raycastTarget = false; // reading face, never a tap thief
_closeButton = MakeButton(_panel.transform, "StoryClose",
    new Vector2(0f, -430f), 220, 76);
_closeButton.GetComponentInChildren<Text>().fontSize = 26;
_closeButton.GetComponentInChildren<Text>().text = CopyTable.Get("feed_page_close");
_closeButton.onClick.AddListener(OnCloseTapped);
}

static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var t = go.GetComponent<Text>();
    t.font = UIFont.Resolve();
    t.fontSize = Mathf.RoundToInt(size);
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    return t;
}

static Button MakeButton(Transform parent, string name, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var img = go.GetComponent<Image>();
    img.color = new Color(0.16f, 0.24f, 0.32f, 1f);
    var btn = go.GetComponent<Button>();
    // One-Graphic-per-GO engine law: Tuanjie 2022.3 refuses a second

```

```

        // Graphic on the same GameObject (GetComponent<Text> returns null,
        // X1192 wave-1 proof), so the label rides a stretch-to-parent
        // child GO instead.
        var labelGo = new GameObject("Label", typeof(RectTransform), typeof(Text));
        labelGo.transform.SetParent(go.transform, false);
        var lrt = (RectTransform)labelGo.transform;
        lrt.anchorMin = Vector2.zero;
        lrt.anchorMax = Vector2.one;
        lrt.offsetMin = Vector2.zero;
        lrt.offsetMax = Vector2.zero;
        var label = labelGo.GetComponent<Text>();
        label.font = UIFont.Resolve();
        label.fontSize = 34;
        label.alignment = TextAnchor.MiddleCenter;
        label.color = Color.white;
        // X1700 AU10: generic tap voice on every programmatic button,
        // registered before the host's semantic listener so the beat
        // lands after the click (order law, census-neutral new key).
        btn.onClick.AddListener(delegate { G16AudioHost.Ensure().PlaySfx("sfx_btn"); });
        return btn;
    }
}
}
// StoryRuntime.cs
// G16 StoryRuntime hub (T022-d1a, M15 Oasis Story Almanac section 6): the
// single-writer mount of the three story rings - ONE PlayGate, ONE
// DramaBuff, ONE ArcState for the whole game. Hosts feed gameplay
// milestones THROUGH the hub (never straight to the rings), the hub
// translates the three evt anchors (evt_chapter_unlock, evt_drama_buff,
// evt_arc_flip) into the story event stream (the footprint-feed View
// slice reads this stream - story history is recorded, not re-derived),
// and it composes the ring-2/3 cut-window factor the bargain host
// applies per duel (the ch1 gift widens every duel window; Hike's
// dive-price comedy widens only his own while he still duels). Zero
// chapters = the bare baseline everywhere - a gain is comfort, never a
// gate (U091). Pure decision layer like the rings: zero RNG, zero clocks.
using System;
using System.Collections.Generic;
using System.Text;
namespace Sy.CrazyEstate
{
    /// <summary>One story event in the stream: which anchor fired, in what
    /// order, carrying which chapter / gain / flip ordinal.</summary>
    public sealed class StoryEventRecord
    {
        public string EventId;
        public int Seq;      // stream order, 1-based, strictly monotone
        public int Payload;  // chapter number / gifting chapter / flip ordinal
    }
    public static class StoryRuntime

```

```

{
    static PlayGate _gate = new PlayGate();
    static DramaBuff _buff = new DramaBuff();
    static ArcState _arc = new ArcState();
    static readonly List<StoryEventRecord> _stream = new List<StoryEventRecord>();
    static int _seq;
    static StoryRuntime()
    {
        Wire();
    }
    // ---- ring-1/2/3 single-writer mounts (hosts read, never replace) ----
    public static PlayGate Gate { get { return _gate; } }
    public static DramaBuff Buff { get { return _buff; } }
    public static ArcState Arc { get { return _arc; } }
    /// <summary>The story event stream (evt anchor translations, append
    /// order). The footprint-feed View slice is the consumer.</summary>
    public static IReadOnlyList<StoryEventRecord> Stream { get { return _stream; } }
    // ---- host feed API ----
    /// <summary>Feed one gameplay milestone: the ring-1 gate latches the
    /// footprint and walks, then the ring-2 gains and ring-3 flips sync
    /// to the walked gate (both idempotent latches inside). Returns the
    /// highest chapter THIS feed unlocked (0 = the armed gate waits -
    /// zero dead ends, keep playing the core loop).</summary>
    public static int Feed(GateFeed feed, double payload = 0.0, int intPayload = 0)
    {
        var unlocked = _gate.Feed(feed, payload, intPayload);
        _buff.SyncFromGate(_gate);
        _arc.SyncFromGate(_gate);
        return unlocked;
    }
    /// <summary>Closed-deal feed (the bargain host's receipt.Accepted
    /// point): the ring-1 first-deal footprint plus the ring-3 viewer
    /// counter in one call - the livestream number reads REAL closed
    /// deals, never invented. listingId &lt; 0 = the unbound guided boot
    /// duel, which is still one real closed deal. The H07 future-city
    /// pre-sale latches the ch8 footprint at the same receipt (one deal
    /// position, two footprint facts - the id space is ListingTable).</summary>
    public static void FeedDealClosed(int listingId)
    {
        Feed(GateFeed.DealClosed, 0.0, listingId);
        if (listingId == PlayGate.FutureCityListingId) Feed(GateFeed.HoldingH07);
        _arc.FeedDealClosed();
    }
    /// <summary>Duel-start feed (the bargain host's listing-bind point):
    /// only Hike duels count toward the ch11 "only you treated me as a
    /// peer" verdict - the warmth is earned by haggling HIM.</summary>
    public static void FeedDuelStart(int agentId)
    {
        if (agentId == ArcState.AgentHike) _arc.FeedHikeDuel();
    }
}

```

```
// ---- ring-2/3 bargain-window composition (host re-computation) ----
/// &summaryComposite cut-window factor the bargain host sets on the
/// sim per duel: the ring-2 ch1 gift widens EVERY duel window; the
/// ring-3 Hike dive-price comedy widens only HIS window while he
/// still duels (after the ch11 noodle exit the factor returns to the
/// gift-only face). Zero chapters = 1.0 identity (bare baseline).&/summary
public static double ComposeCutWindowFactor(int agentId)
{
    var factor = 1.0 + _buff.BargainWindowBonusPct() / 100.0;
    if (agentId == ArcState.AgentHike) factor *= _arc.HikeCutWindowFactor();
    return factor;
}
/// &summaryHub fingerprint: the three ring fingerprints plus the
/// stream trace - the CH15 double-run determinism face.&/summary
public static string HubFingerprint()
{
    var sb = new StringBuilder();
    sb.Append(_gate.Fingerprint()).Append('#');
    sb.Append(_buff.Fingerprint()).Append('#');
    sb.Append(_arc.Fingerprint()).Append('#');
    foreach (var e in _stream)
        sb.Append(e.Seq).Append(':').Append(e.EventId).Append(':').Append(e.Payload).Append(';');
    return sb.ToString();
}
// ---- test seam (Editor checks only; production never resets) ----
/// &summaryFresh hub for the Editor check law (CH15): recreates the
/// three rings, rewires the anchor translations and clears the
/// stream. Production code must never call this.&/summary
public static void ResetForChecks()
{
    _gate = new PlayGate();
    _buff = new DramaBuff();
    _arc = new ArcState();
    _stream.Clear();
    _seq = 0;
    Wire();
}
static void Wire()
{
    _gate.OnChapterUnlocked += ch => Append(PlayGate.ChapterUnlockEventId, ch);
    _buff.OnGainMounted += ch => Append(DramaBuff.GainMountEventId, ch);
    _arc.OnArcFlip += f => Append(ArcState.ArcFlipEventId, f);
}
static void Append(string eventId, int payload)
{
    _seq++;
    _stream.Add(new StoryEventRecord { EventId = eventId, Seq = _seq, Payload = payload });
}
}
```